



操作系统实验报告

Lab 2: Threads and Synchronization

姓 名：李聪

学 号：22920212204124

院 系：信息学院人工智能系

专 业：人工智能

年 级：2021 级

指导教师：曹冬林

2023 年 11 月 30 日

一、实验目的	4
二、实验要求	4
1.1 实现锁机制和条件变量	4
2.2 实现一个线程安全的表结构	5
2.3 实现一个大小受限的缓冲区	5
2.4 （附加实验）实现优先级调度程序	6
2.5 实验 2 的一些注意事项	6
三、实验流程	7
3.1 实现锁机制和条件变量	7
3.1.1 编写 synch-sleep.h 和 synch-sleep.cc 文件	7
3.1.2 编写 synch-sem.h 和 synch-sem.cc 文件	8
3.1.3 修改 dllist.h 和 dllist.cc 文件	9
3.1.4 修改 threadtest.cc 文件	10
3.1.5 运行测试	11
3.2 实现一个线程安全的表结构	11
3.2.1 编写 Table.h 和 Table.cc 文件	11
3.2.2 修改 threadtest.cc 文件	13
3.2.3 运行测试	13
3.3 实现一个大小受限的缓冲区	13
3.3.1 编写 BoundedBuffer.h 和 BoundedBuffer.cc 文件	13
3.3.2 修改 threadtest.cc 文件	15
3.3.3 运行测试	15
3.4 实现优先级调度程序	15
3.4.1 修改 thread.h 和 thread.cc 文件	15
3.4.2 修改 scheduler.cc 文件	16
3.4.3 修改 threadtest.cc 文件	16
3.4.4 运行测试	16
3.5 利用优先级调度程序观察不同优先级的生产者消费者问题	16
3.5.1 修改 threadtest.cc 文件	16
3.5.3 运行测试	16
四、测试及分析	16
4.1 验证锁机制和条件变量的正确性	17
4.1.1 利用实验 1 中的测试函数和 YIELD 函数验证	17
4.1.2 利用 -rs 种子验证	18
4.2 验证线程安全的表结构实现的正确性	21
4.2.1 验证单线程下的表结构各种操作的正确性	21
4.2.3 验证多线程下的表结构各种操作的正确性	22
4.3 验证大小受限的缓冲区实现的正确性	24
4.4 验证优先级调度程序的正确性	27
4.5 利用优先级调度程序观察不同优先级的生产者消费者问题	28
五、实验总结	29
5.1 信号量实现条件变量错误	29
5.2 条件变量的 Wait 函数中，“waitNum++”位置错误	29
5.3 Write 和 Read 函数实现错误	29

六、实验心得	29
七、附件	29
附件 1 synch-sleep.h	29
附件 2 synch-sleep.cc	33
附件 3 synch-sem.h.....	37
附件 4 synch-sem.cc.....	41
附件 5 dllist.h	45
附件 6 dllist.cc	45
附件 7 Table.h.....	52
附件 8 Table.cc	53
附件 9 BoundedBuffer.h	54
附件 10 BoundedBuffer.cc.....	55
附件 11 threadtest.cc	56
附件 12 main.cc	65

一、实验目的

1. 实现 Nachos 的同步机制：锁和条件变量；
2. 利用这些同步机制实现一些工具类；
3. 了解默认的 Nachos 调度程序，并基于线程优先级的调度程序。

二、实验要求

1.1 实现锁机制和条件变量

互斥体和条件变量的公共接口在 `synch.h` 中定义，其中包括关于原语语义的重要注释。条件变量接口在某些方面很笨拙，但请接受 `synch.h` 定义的接口。您的第一个任务是在 `synch.h` 中为这些类定义私有数据，并在 `synch.cc` 中实现接口。查看 `SynchList` 以了解如何使用互斥体和条件变量的同步原语。您需要在 `synch.h` 和 `synch.cc` 文件的两个不同版本中编写这些同步原语的两个不同实现。通过（最坏的情况下）移动这些文件并重新编译 `nachos`，您应该能够从一个版本切换到另一个版本。

以下是更详细的具体步骤和要求：

（1）使用睡眠/唤醒原语（`Thread::sleep` 和 `Scheduler::ReadyToRun` 原语）实现锁定和条件变量。有必要在战略点暂时禁用中断，以消除不适时中断或非自愿上下文切换的可能性。特别是，`Thread::Sleep` 要求您在调用中断之前禁用中断。但是，在没有必要禁用中断时，您可能会因保持中断处于禁用状态而失去分数。禁用中断是一种钝器，除非必要，否则应避免使用。

（2）使用信号量作为唯一的同步原语来实现锁和条件变量。这一次没有必要（或允许）在代码中禁用中断：信号量原语根据实现信号量抽象的需要禁用中断，您现在可以使用它作为同步的充分“立足点”。警告：这部分作业看似简单，但实际上是最微妙和困难的。特别是，条件变量的解决方案应确保信号不会影响后续等待。

（3）从实验 1 中修改 `DLList` 类，使其使用同步原语，以确保即使多个线程使用该列表，该列表仍能持续更新。您可能希望查看 `SynchList` 类（在 `threads/SynchList.h` 和 `threads/SynchList.cc` 中）以了解如何执行此操作。为了演示代码的工作原理，如实验 1 所示，创建一个类似于文件 `threadtest.cc` 的驱动程序文件，该文件调用 `DLList` 类的同步版本。对

`threads/main.cc` 进行更改，以便执行 `nachos` 命令会导致执行新驱动程序文件中的函数，而不是文件 `ThreadTest.cc` 中的函数 `ThreadTest`。使用 `DLList` 的同步版本测试锁和条件变量的两个版本。

2.2 实现一个线程安全的表结构

实现一个线程安全的表类，它存储由 `[0..size-1]` 范围内的整数索引的非类型化对象指针的集合。您可以在以后的实验中使用 `Table` 来实现进程、线程、内存页帧、打开文件等的内部操作系统表。`Table` 具有以下方法，这些方法在课程目录（子目录 `aux`）中提供的头文件 `Table.h` 中定义：

`Table(int size)` — 创建一个表以容纳最多大小的条目。

`int Alloc (void* object)` — 为对象分配一个表槽，返回已分配项的索引。如果没有可用的表插槽，则返回错误（`-1`）。

`void* Get (int index)` — 从索引处的表槽中检索对象，如果未分配，则返回 `NULL`。

`void Release (int index)` — 在索引处释放表槽。

2.3 实现一个大小受限的缓冲区

这是一个称为有界生产者 / 消费者的经典同步问题。基于 `*/aux/BoundedBuffer.h` 中的定义，实现一个线程安全的 `BoundedBuffer` 类。

`BoundedBuffer(int maxsize)` — 创建一个最多容纳 `maxsize` 字节的有界缓冲区。

`void Read(void*data, int size)` — 从缓冲区读取大小字节，直到有足够的字节完全满足请求为止。从地址数据开始将字节复制到内存中。

`void Write (void* data, int size)` — 将大小字节写入缓冲区，直到有足够的空间完全满足请求为止。从地址数据开始从内存复制字节。

将在 `Unix` 实验室中使用基本的缓冲区（`IPC`）管道来实现 `ED5` 通信机制。基本思想是管道或 `BoundedBuffer` 将数据从生产者线程（调用写入）传递到消费者线程（调用读取）。使用者通过写入接收放在缓冲区中的字节，其顺序与生产者写入的字节顺序相同。如果生产者生成数据太快（即缓冲区溢出超过 `maxsize` 字节），则写入会使生产者处于休眠状态，直到消费者能够从缓冲区捕获并读取一些数据，从而释放空间。如果使用者读取数据太快（即缓冲区清空），

则读取会使使用者进入睡眠状态，直到生产者能够赶上并生成更多字节。请注意，对于哪些线程调用 **Read** 和哪些线程调用 **Write** 没有限制。您的实现不应该假设只有两个线程使用它，或者调用线程充当生产者和消费者的固定角色。如果给定的 **BoundedBuffer** 被多个线程使用，那么您应该注意保持读写请求的原子性。也就是说，由给定写入操作写入的数据永远不应该与来自其他写入操作的数据交叉发送到读取器。即使写入程序和/或读取器由于缓冲区填满或耗尽而被迫阻塞，该不变量也应保持不变。

2.4 （附加实验）实现优先级调度程序

跟踪 `threads/thread.cc` 、 `threads/scheduler.h` 和 `threads/scheduler.cc` 中的代码，以了解默认 **nachos** 调度程序如何选择要运行的下一个线程（无论是线程显式生成 CPU 时，还是当时间片完成时）。要实现基于优先级的非抢占式调度程序，请修改线程调度程序以始终返回优先级最高的线程。您可能希望将 `scheduler.h` 和 `scheduler.cc` 复制到具有不同名称的文件中，并修改这些副本而不是原始副本。您还需要为线程创建一个新的构造函数，以获取另一个参数——线程的优先级。请保持旧的构造函数不变，因为您需要它来实现向后兼容性。你可以假设有一个小的，固定数量的优先级-对于这个任务，你只需要两个。通过使用调度器实现运行有界缓冲区实现来测试调度器实现。更改生产者线程和消费者线程的相对优先级会对输出产生影响吗？例如，当一个生产者的优先级高于另一个生产者时，两个生产者和一个消费者会发生什么情况？如果两个生产者的优先级相同，但消费者的优先级较高，该怎么办？要获得所需的行为，请确保在生产者和消费者代码中调用了 `Thread::Yield`（仅针对实验的这一部分）。

2.5 实验 2 的一些注意事项

锁和条件变量的实现应使用第 2.2 节（请参阅跟踪和调试 **Nachos** 程序）中所述的断言检查，以强制执行正确行为所需的任何使用约束。例如，对信号和等待的每次调用都会传递一个相关的互斥锁；如果给定的条件变量与多个互斥体一起使用，会出现什么问题？如果持锁人再次尝试获取持有的锁，会发生什么情况？如果线程试图释放它不持有的锁，该怎么办？这些断言检查是此分配的价值点，它们将在以后的分配中为您省去麻烦。

您还需要考虑其他使用问题。例如，如果调用方试图删除一个互斥对象或条件变量对象，而该对象上有阻塞的线程，那么您的实现应该怎么做？您应该能够解释为什么您的实现是正确的（例如，如果我们将 `yield` 放在 `X` 行和 `Y` 行之间会发生什么），并在各种使用场景下对其行为（公平性、饥饿等）进行评论。

警告： Nachos 条件变量接口很难看，因为它在每次调用 `Wait` 或 `Signal` 时都会传递相关的互斥体，而不是在条件变量构造函数中只绑定一次互斥体。这意味着您必须添加代码来记住第一次调用时要等待或发出信号的互斥量，以便在后续调用中验证正确的用法。但请不要搞错：每个条件变量只与一个互斥体一起使用，如 `synch.h` 中所述。确保你理解为什么这是如此重要。

警告： 信号量的定义不能保证在 `V` 中唤醒的线程在另一个线程调用 `P` 之前有机会运行。特别是，信号量的 Nachos 实现不能保证这种行为。也就是说，如果计数从零变为一，`V` 会增加计数并唤醒阻塞的线程，但唤醒的线程有责任在 `P` 中唤醒后再次减少计数。如果另一个线程先调用 `P`，那么它可能会使用“为”被唤醒线程“准备”的计数，这将导致被唤醒线程返回睡眠状态并等待另一个 `V`。

注意： 对于调试，可以使用 `-s` 调试标志。但是，当前没有带 `s` 的 `DEBUG` 标志的调试语句，因此需要向代码中添加一些。参见第 2.2 节（参见 Nachos 程序的跟踪和调试）。

三、实验流程

3.1 实现锁机制和条件变量

3.1.1 编写 `synch-sleep.h` 和 `synch-sleep.cc` 文件

利用禁用中断来实现锁和条件变量操作的原子性，消除不适时中断或非自愿上下文切换的可能性，在这些操作开始前，都设置中断标志位为禁止中断，在操作完成后，恢复原来的中断标志位状态：

```
IntStatus oldLevel = interrupt->SetLevel(IntOff); // 中断禁用
...
(void)interrupt->SetLevel(oldLevel); // 恢复中断标志状态
```

1) 锁的主要操作有请求 `Acquire` 和释放 `Release`。

a. 进程请求锁时，要先判断锁是否已被其他进程持有，若已被持有，则当前进程应被阻塞，进入阻塞队列 `queue`，等待其他进程释放锁而并非被唤醒（`Release` 中 `ReadyToRun`），唤醒后再次请求锁，直到锁无人拥有，则获得锁：

```
while (lockOwner != NULL)
{
    queue->Append((void *) currentThread);
    currentThread->Sleep();
}
```

```
lockOwner = currentThread;
```

b. 进程释放锁时,要唤醒一个因不能获得锁而阻塞的进程,即从阻塞队列 `queue` 中删除一个进程(若存在),并放入就绪队列,注意要将 `lockOwner` 置为 `NULL`,表示锁无人拥有:

```
Thread *thread = (Thread *)queue->Remove();
if (thread != NULL) scheduler->ReadyToRun(thread);
if (currentThread == lockOwner) lockOwner = NULL;
```

【注】被唤醒的进程只是进入就绪队列,并不是立刻就运行,而是在就绪队列中等待 CPU 的调度,这符合实验的要求。

2) 条件变量的主要操作有 `Wait`、`Signal` 和 `Broadcast`。

a. 条件变量一般和锁一起使用,在拥有锁后,但不满足某种条件时,需要使用 `Wait` 来等待这一条件被满足。所以在 `Wait` 操作中,应该先释放锁,进入阻塞队列 `queue`,等待 `Signal` 或 `Broadcast` 发出的条件满足的信号而被唤醒(进入就绪队列),然后重新申请锁,以便继续后续运行:

```
conditionLock->Release(); // 释放锁
queue->Append((void *) currentThread);
currentThread->Sleep(); // 放弃 CPU
conditionLock->Acquire(); // 请求锁
```

【注】先释放锁的目的是以便其他进程可以获得锁,执行某些操作来满足 `Wait` 需要的条件,若不先释放锁,则其他进程无法获得锁,进程永远无法被唤醒。

b. `Signal` 发送条件满足的信号,唤醒等待相应条件满足的一个进程,进入就绪队列等待调度:

```
Thread *thread = (Thread *)queue->Remove();
if (thread != NULL) scheduler->ReadyToRun(thread);
```

c. `Broadcast` 和 `Signal` 类似,但是唤醒的是所有等待相应条件满足的进程,用 `while` 循环来实现:

```
Thread *thread = (Thread *)queue->Remove();
while (thread != NULL)
{
    scheduler->ReadyToRun(thread);
    thread = (Thread *)queue->Remove();
}
```

3.1.2 编写 [synch-sem.h](#) 和 [synch-sem.cc](#) 文件

1) 利用信号量实现锁,可以将锁看成一个初始值为 1 的信号量,获取锁 `Acquire` 就是信号量的 P 操作,释放锁 `Release` 就是信号量的 V 操作,要注意 `lockOwner` 的变化:

```
void Lock::Acquire()
{
    lock->P();
    lockOwner = currentThread;
}
void Lock::Release()
{
    if(currentThread == lockOwner)
```



```

    {
        lockOwner = NULL;
        lock->V();
    }
}

```

2) 利用信号量实现条件变量，也可以将条件看成一个初始值为 0 的信号量，Wait 对应着 P 操作，Signal 和 Broadcast 对应着 V 操作。初始值为 0 的原因是一旦条件不满足，调用 Wait 执行 P 操作就会被阻塞。但仅仅是这样，实现还存在着问题，因为进程的行为可以很经常的满足某种条件而调用 Signal 或 Broadcast，如果每次都执行 V 操作，则在条件不被满足时，由于之前 V 操作使信号量增大，调用 Wait 不会被阻塞，所以需要记录等待条件的进程的数量（waitNum），只有在有等待条件的进程时，执行 V 操作，这样也能便于 Broadcast 的实现：

```

void Condition::Wait(Lock *conditionLock)
{
    conditionLock->Release();
    condition->P();
    ++ waitNum;
    conditionLock->Acquire();
}

void Condition::Signal(Lock *conditionLock)
{
    if(waitNum > 0)
    {
        -- waitNum;
        condition->V();
    }
}

void Condition::Broadcast(Lock *conditionLock)
{
    while(waitNum > 0)
    {
        -- waitNum;
        condition->V();
    }
}

```

3.1.3 修改 [dllist.h](#) 和 [dllist.cc](#) 文件

1) 给 dllist 类添加两个私有成员函数：

```

Lock *lock;           // 互斥锁
Condition *listEmpty; // 非空条件

```

2) 修改构造函数添加新的成员变量的定义：

```

lock = new Lock("ListLock");
listEmpty = new Condition("ListEmptyCond");

```

3) 分析可知多线程对链表的插入和删除需要互斥的访问链表，以及删除操作需

要满足非空条件，而插入操作可以提供非空条件。

所以需要给 `dllist` 类的所有插入和删除操作添加互斥锁，即在操作开始前先申请锁（`lock->Acquire()`），操作完成后释放锁（`lock->Release()`）。

在插入操作完成后释放锁前，应该发出满足非空条件的信号（`listEmpty->Signal(lock)`）：

```
lock->Acquire();
// 插入操作
...
listEmpty->Signal(lock);
lock->Release();
```

在删除操作上锁后，若链表为空，则应该发出等待非空条件的信号（`listEmpty->Wait(lock)`）：

```
lock->Acquire();
while (IsEmpty()) listEmpty->Wait(lock);
// 删除操作
...
lock->Release();
```

3.1.4 修改 [threadtest.cc](#) 文件

1) 编写三个线程创建函数，为后续创建线程服务：

- `func` 是指向线程调用的函数的指针。
- “`kind id`”是线程的名字。
- `MinPriority` 是优先级的最小值，采取随机设定优先级。
- `priority` 是线程的优先级。

这三个函数作用是创建一个线程来执行 `func` 函数，`makeThread` 函数不传入优先级 `priority` 参数，创建的是默认优先级的线程，而传入优先级 `priority` 参数，则根据 `priority` 创建线程；`makeRandThread` 函数是给定优先级的范围 `[0, MinPriority)` 随机初始化线程的优先级。

```
void makeRandThread(void(*func)(int arg), char *kind, int id, int
MinPriority)
{
    int n1 = strlen(kind), n2 = getIntBit(id);
    char* name = new char[n1 + n2 + 2];
    name[0] = '\0';
    strcat(name, kind);
    sprintf(name + n1, " %d", id);
    Thread *thread = new Thread(name, rand() % MinPriority);
    thread->Fork(func, id);
}

void makeThread(void(*func)(int arg), char *kind, int id, int priority)
{
    int n1 = strlen(kind), n2 = getIntBit(id);
    char* name = new char[n1 + n2 + 2];
    name[0] = '\0';
    strcat(name, kind);
```

```

    sprintf(name + n1, " %d", id);
    Thread *thread = new Thread(name, priority);
    thread->Fork(func, id);
}
void makeThread(void(*func)(int arg), char *kind, int id)
{
    int n1 = strlen(kind), n2 = getIntBit(id);
    char* name = new char[n1 + n2 + 2];
    name[0] = '\\0';
    strcat(name, kind);
    sprintf(name + n1, " %d", id);
    Thread *thread = new Thread(name);
    thread->Fork(func, id);
}

```

2) 添加 `test2` 函数，由命令行参数指定执行插入操作的线程的数量，执行删除操作的线程的数量，以及它们分别插入和删除的元素个数。

3.1.5 运行测试

对于两种实现的验证内容相同，每次验证只需要替换对应 `synch.h` 和 `synch.cc` 中的内容。

进行以下测试：

(a) 利用实验 1 中留下的“`currentThread->Yield()`”，尝试每一种可能，验证实现的正确性。

(b) 调用 `test2` 函数，使用 `-rs` 命令行参数，使得进程可以实现随机切换。

3.2 实现一个线程安全的表结构

3.2.1 编写 `Table.h` 和 `Table.cc` 文件

1) `Table` 类的定义如下：

```

class Table
{
public:
    Table(int size);
    ~Table();
    int Alloc (void* object);
    void* Get (int index);
    void Release (int index);
private:
    int TableSize;
    void **table;
    Semaphore* lock;
};

```

`table` 是表的主体。

`TableSize` 是表的大小。

`lock` 是初始值为 1 的信号量，用来实现对 `table` 的互斥访问，可以看成锁。

2) `Table` 的构造函数：

```

Table::Table(int size)

```

```
{
    TableSize = size;
    table = new void*[size];
    for(int i = 0; i < size; ++ i) table[i] = NULL;
    lock = new Semaphore("TableLock", 1);
}
```

3) Table 的析构函数:

```
Table::~~Table()
{
    delete table;
    delete lock;
}
```

4) Alloc 函数按顺序遍历 table 表, 将 object 插入到第一个空位, 返回插入的位置, 若 table 表已满, 则返回 -1:

```
int Table::Alloc (void* object)
{
    lock->P();
    for(int i = 0; i < TableSize; ++ i)
        if(table[i] == NULL)
        {
            table[i] = object;
            lock->V();
            return i;
        }
    lock->V();
    return -1;
}
```

5) Get 函数根据索引 index 返回 table 对应位置的内容, 若为空, 返回 NULL:

```
void* Table::Get (int index)
{
    lock->P();
    if(index >= 0 && index < TableSize && table[index] != NULL)
    {
        lock->V();
        return table[index];
    }
    lock->V();
    return NULL;
}
```

6) Release 函数根据索引清空 table 对应位置的内容, 即赋值为 NULL:

```
void Table::Release (int index)
{
    lock->P();
    if(index >= 0 && index < TableSize) table[index] = NULL;
}
```

```
lock->V();  
}
```

【注】Table 的三个操作都应该加上锁以保证线程安全，由于题目没有特别要求所以不设置条件变量。

3.2.2 修改 [threadtest.cc](#) 文件

- 1) 添加 test3 函数，用来验证单线程环境下的各种操作的正确性。
- 2) 添加 test4 函数，用来验证多线程环境下 Table 类实现的正确性，table 的三个操作分别由三种线程完成，三种线程的数量由命令行参数指定。

3.2.3 运行测试

3.3 实现一个大小受限的缓冲区

3.3.1 编写 [BoundedBuffer.h](#) 和 [BoundedBuffer.cc](#) 文件

- 1) BoundedBuffer 类的定义如下：

```
class BoundedBuffer  
{  
public:  
    BoundedBuffer(int maxsize);  
    ~BoundedBuffer();  
    void Read (void* data, int size);  
    void Write (void* data, int size);  
private:  
    char *buffer;  
    int maxSize;  
    int head;  
    int tail;  
    Semaphore* writeLock;  
    Semaphore* readLock;  
    Semaphore* use;  
    Semaphore* remain;  
};
```

buffer 是缓冲区的主体，大小固定的循环队列；

maxSize 是缓冲区的大小；

head 是缓冲区内容的头位置；

tail 是缓冲区内容的尾位置；

writeLock 是初始值为 1 的信号量，用来实现写进程之间的互斥，即“同时”只能有一个进程往缓冲区中写入数据；

readLock 是初始值为 1 的信号量，用来保证读进程之间的互斥即“同时”只能有一个进程从缓冲区中读出数据；

use 是初始值为 0 的信号量，表示缓冲区已使用空间的大小；

remain 是初始值为 maxsize 的信号量，表示缓冲区剩余空间的大小；

- 2) BoundedBuffer 的构造函数：

```
BoundedBuffer::BoundedBuffer(int maxsize)  
{  
    buffer = new char[maxsize];  
    maxSize = maxsize;
```

```

    head = tail = 0;
    lock = new Semaphore("BufferLock", 1);
    used = new Semaphore("UsedSpace", 0);
    remaining = new Semaphore("RemainingSpace", maxsize);
    buffer = new char[maxsize];
}

```

3) BoundedBuffer 的析构函数:

```

BoundedBuffer::~~BoundedBuffer()
{
    delete buffer;
    delete lock;
    delete used;
    delete remaining;
}

```

4) Read 函数要获得 readLock 后,才可以执行后续操作,重复读取操作 size 次,每次先获取一个 use 信号量,表示自己要读取一个字节,再对 buffer 进行操作,完成读取操作后,释放一个 remai 信号量,完成所有读取操作后,要释放 readLock:

```

void BoundedBuffer::Read(void *data, int size)
{
    char *out = (char *) data;
    readLock->P();
    for(int i = 0; i < size; ++ i)
    {
        use->P();
        out[i] = buffer[head];
        head = (head + 1) % maxSize;
        remain->V();
    }
    readLock->V();
}

```

5) Write 函数要获得 writeLock 后,才可以执行后续操作,重复写入操作 size 次,每次先获取一个 remain 信号量,表示自己要写入一个字节,再对 buffer 进行操作,完成写入操作后,释放一个 use 信号量,完成所有读取操作后,要释放 writeLock:

```

void BoundedBuffer::Write(void *data, int size)
{
    char *in = (char *) data;
    writeLock->P();
    for(int i = 0; i < size; ++ i)
    {
        remain->P();
        buffer[tail] = in[i];
        tail = (tail + 1) % maxSize;
    }
    writeLock->V();
}

```

```

        use->V();
    }
    writeLock->V();
}

```

【注】Write 和 Read 使用不同的锁，即保证在 Write 或 Read 完全写入或读取自己所需内容前，不会让其他 Write 或 Read 写入或读取，但 Write 完全写入前，Read 可以先读取部分内容，这符合实验对 Write 和 Read 行为的描述。

3.3.2 修改 [threadtest.cc](#) 文件

1) 添加 test5 函数，由命令行参数指定生产者和消费者的数量以及对应操作的字节数。

3.3.3 运行测试

3.4 实现优先级调度程序

3.4.1 修改 thread.h 和 thread.cc 文件

1) 给 Thread 类添加私有成员变量优先级 priority:

```
int priority; // 进程的优先级
```

2) 给 Thread 类添加共有成员函数:

```
Thread(char* debugName, int value){ name = debugName, priority = value;}
int getPriority() { return priority; }
void setPriority(int value) { priority = value; }

```

3) 修改 Thread 类的构造函数，使其兼容优先级，默认初始化为 0:

```
Thread::Thread(char *threadName)
{
    name = threadName;
    priority = 0; // 默认初始化为 0
    stackTop = NULL;
    stack = NULL;
    status = JUST_CREATED;
#ifdef USER_PROGRAM
    space = NULL;
#endif
}

```

4) 添加可以按需初始化优先级的构造函数，在上面的构造函数的基础上，添加优先级的初始化，以保证添加的构造函数的兼容性:

```
Thread::Thread(char *threadName, int value)
{
    name = threadName;
    priority = value;
    stackTop = NULL;
    stack = NULL;
    status = JUST_CREATED;
#ifdef USER_PROGRAM
    space = NULL;
#endif
}

```

3.4.2 修改 scheduler.cc 文件

线程的调度都是先由 ReadyToRun 放入就绪队列，再由 Run 运行，而在发生线程的切换时，默认是从就绪队列的头，取出一个线程，作为下一时刻运行的线程，所以要实现优先级调度，只要修改 Scheduler 类的 ReadyToRun 函数，使得可以将进程按优先级插入就绪队列，即将 Append 改成 SortedInsert:

```
void Scheduler::ReadyToRun(Thread *thread)
{
    DEBUG('t', "Putting thread %s on ready list.\n", thread->getName());

    thread->setStatus(READY);
    // readyList->Append((void *)thread);
    readyList->SortedInsert((void *)thread, thread->getPriority());
}
```

3.4.3 修改 threadtest.cc 文件

1) 添加 test6 函数，测试进程调度的顺序。函数会创建 n 个线程（由命令行参数指定），每个线程只有一个操作，输出当前运行的线程的名字和优先级。

```
void thread6(int witch)
{
    printf("%s %d begin to run.\n", currentThread->getName(),
currentThread->getPriority());
}
void test6()
{
    int n = fargv[0];
    srand(time(0));
    for(int i = 0; i < n; ++ i)
        makeRandThread(thread6, "mythread", i, 100);
}
```

3.4.4 运行测试

3.5 利用优先级调度程序观察不同优先级的生产者消费者问题

3.5.1 修改 threadtest.cc 文件

1) 添加 test7 函数，函数中创建三个进程，其中两个是生产者“Writer 0”和“Writer 1”，一个是消费者“Reader 0”。函数需要七个参数，按顺序分别是缓冲区的大小、“Writer 0”的优先级、“Writer 0”调用 Write 的次数、“Writer 1”的优先级、“Writer 1”调用 Write 的次数、“Reader 0”的优先级、“Reader 0”调用 Reader 的次数。为了简便，又不失一般性，Write 和 Read 每次都是写入或读取两个字节。

3.5.3 运行测试

四、测试及分析

命令: ./nachos -q <testNum> [<fargvs>] -d [<flags>] -y [<place>]
-rs [<seed>]

-q 是和测试函数有关的命令行参数:

<testNum>是整型，指定调用的测试函数;

<fargs>是整型数组，指定调用的测试函数的参数。

-d 是和调试有关的命令行参数：

<flags>是字符串，每个字符指定一种调试信息，我设定的是‘e’字符，与线程调度相关的是‘t’字符。

-y 是和中断有关的命令行参数：

<place>是整型，指定中断的位置，和其一样的 YIELD 参数，会产生中断。

-rs 是和随机线程切换有关的命令行参数：

<seed>是种子。

4.1 验证锁机制和条件变量的正确性

4.1.1 利用实验 1 中的测试函数和 YIELD 函数验证

命令：./nachos -q 1 t n -d e -y p

该测试调用的是 test1 函数，有两个参数：

- t--线程的数量
- n--每个线程插入和删除元素的个数

将 synch-sleep.h 和 synch-sleep.cc 复制到 synch.h 和 synch.cc 中，编译运行，尝试所有中断点，可以发现不再像实验 1 中那样报错，都正常运行。对于 synch-sem.h 和 synch-sem.cc 执行同样的操作。，结果也是不再报错，正常运行。

例子：

```
1 $ ./nachos -q 1 2 4 -d e -y 12
2 main: make thread thread 0
3 main: make thread thread 1
4 thread 0 is running.
5 thread 0: insert key:71, item:0x8c7e1a8.
6 thread 0: insert key:94, item:0x8c84258.
7 thread 0: insert key:56, item:0x8c84280.
8 thread 0: insert key:46, item:0x8c842a8.
9 thread 0 yield in 12.
10 thread 1 is running.
11 thread 0: remove key:46, item:0x8c842a8.
12 thread 0 yield in 12.
13 thread 0: remove key:56, item:0x8c84280.
14 thread 0 yield in 12.
15 thread 0: remove key:71, item:0x8c7e1a8.
16 thread 0 yield in 12.
17 thread 0: remove key:94, item:0x8c84258.
18 thread 0 is finish.
19 List is empty!
20 thread 1: insert key:59, item:0x8c842d0.
21 thread 1: insert key: 7, item:0x8c84230.
22 thread 1: insert key:79, item:0x8c842e0.
23 thread 1: insert key:80, item:0x8c780c0.
24 thread 1 yield in 12.
```

```

25 thread 1: remove key: 7, item:0x8c84230.
26 thread 1 yield in 12.
27 thread 1: remove key:59, item:0x8c842d0.
28 thread 1 yield in 12.
29 thread 1: remove key:79, item:0x8c842e0.
30 thread 1 yield in 12.
31 thread 1: remove key:80, item:0x8c780c0.
32 thread 1 is finish.
33 List is empty!

```

分析:

第 1 行是命令指定中断点再 12 号位置, 即 Remove 函数中。

主线程正常创建了两个线程 (第 2-3 行), 转到线程 0;

线程 0 正常执行插入操作 (第 4-8 行), 在第一次删除的位置, 可以看到放弃了 CPU (第 9 行), 转到线程 1;

线程 1 开始运行 (第 10 行), 由于锁的存在, 线程 1 不能对链表操作, 进程阻塞, 等待唤醒, 所以又回到了线程 0;

线程 0 继续删除操作 (第 11 行), 执行完后, 释放锁, 唤醒线程 1 进入就绪队列, 但是并没有放弃 CPU, 继续执行下一个删除操作。又重新获得了锁, 再次主动放弃 CPU (第 12 行), 转到线程 1;

线程 1 依旧没有锁, 无法操作链表, 阻塞, 转到线程 0 (第 13 行), 如此往复。

结论:

从上述分析, 可以看出锁确实保证了线程对于链表的互斥访问, 对于链表的插入删除操作, 都需要先获得锁, 这验证了锁机制的实现是正确的。

4.1.2 利用-rs 种子验证

将 synch-sleep.h 和 synch-sleep.cc 复制到 synch.h 和 synch.cc 中, 编译运行 ./nachos -q t1 n1 t2 n2 -d et -rs seed, 通过尝试多个-rs 种子和多种不同的参数, 都正常运行, 验证锁和条件变量的正确性。对于 synch-sem.h 和 synch-sem.cc 执行同样的操作, 结果也是不再报错, 正常运行。

该测试运行的是 test2 函数, 有四个参数:

- t1 -- 执行插入的线程数量
- n1 -- 每个线程插入的元素个数
- t2 -- 执行删除的线程数量
- n1 -- 每个线程删除的元素个数

例子:

```

1 $ ./nachos -q 2 2 4 1 8 -d et -rs 1000
2 Forking thread "Insert 0" with func = 0x804b1f4, arg = 0
3 Putting thread Insert 0 on ready list.
4 main: make thread Insert 0 73
5 Forking thread "Insert 1" with func = 0x804b1f4, arg = 1
6 Putting thread Insert 1 on ready list.
7 main: make thread Insert 1 14
8 Forking thread "Remove 0" with func = 0x804b182, arg = 0
9 Putting thread Remove 0 on ready list.

```

```
10  main: make thread Remove 0 5
11  Finishing thread "main"
12  Sleeping thread "main"
13  Switching from thread "main" to thread "Remove 0"
14  Remove 0 is running.
15  Sleeping thread "Remove 0"
16  Switching from thread "Remove 0" to thread "Insert 1"
17  Insert 1 is running.
18  Putting thread Remove 0 on ready list.
19  Insert 1: insert key:23, item:0x8d2e270.
20  Insert 1: insert key:60, item:0x8d34320.
21  Insert 1: insert key:45, item:0x8d34348.
22  Insert 1: insert key:69, item:0x8d34370.
23  Insert 1 is finish.
24  Finishing thread "Insert 1"
25  Sleeping thread "Insert 1"
26  Switching from thread "Insert 1" to thread "Remove 0"
27  Now in thread "Remove 0"
28  Deleting thread "Insert 1"
29  Yielding thread "Remove 0"
30  Putting thread Remove 0 on ready list.
31  Switching from thread "Remove 0" to thread "Insert 0"
32  Insert 0 is running.
33  Sleeping thread "Insert 0"
34  Switching from thread "Insert 0" to thread "Remove 0"
35  Now in thread "Remove 0"
36  Putting thread Insert 0 on ready list.
37  Remove 0: remove key:23, item:0x8d2e270.
38  Remove 0: remove key:45, item:0x8d34348.
39  Remove 0: remove key:60, item:0x8d34320.
40  Remove 0: remove key:69, item:0x8d34370.
41  Sleeping thread "Remove 0"
42  Switching from thread "Remove 0" to thread "Insert 0"
43  Now in thread "Insert 0"
44  Yielding thread "Insert 0"
45  Putting thread Remove 0 on ready list.
46  Insert 0: insert key:25, item:0x8d342f8.
47  Insert 0: insert key:76, item:0x8d28220.
48  Insert 0: insert key:53, item:0x8d28230.
49  Yielding thread "Insert 0"
50  Putting thread Insert 0 on ready list.
51  Switching from thread "Insert 0" to thread "Remove 0"
```

```

52 Now in thread "Remove 0"
53 Remove 0: remove key:25, item:0x8d342f8.
54 Remove 0: remove key:53, item:0x8d28230.
55 Remove 0: remove key:76, item:0x8d28220.
56 Remove 0: remove key:90, item:0x8d28240.
57 Remove 0 is finish.
58 Finishing thread "Remove 0"
59 Sleeping thread "Remove 0"
60 Switching from thread "Remove 0" to thread "Insert 0"
61 Now in thread "Insert 0"
62 Deleting thread "Remove 0"
63 Insert 0: insert key:90, item:0x8d28240.
64 Insert 0 is finish.
65 Finishing thread "Insert 0"
66 Sleeping thread "Insert 0"
67 No threads ready or runnable, and no pending interrupts.
68 Assuming the program completed.
69 Machine halting!
70
71 Ticks: total 545, idle 25, system 520, user 0
72 Disk I/O: reads 0, writes 0
73 Console I/O: reads 0, writes 0
74 Paging: faults 0
75 Network I/O: packets received 0, sent 0
76
77 Cleaning up...

```

分析:

主线程创建了三个线程“Insert 0”、“Insert 1”、“Remove 0”，优先级分别为 73、14、5（[第 4、7、10 行](#)），优先级小的先运行，即“Remove 0”先运行（[第 13 行](#)）。

“Remove 0”由于链表为空，不满足非空条件，被阻塞，等待唤醒（[第 15 行](#)），转到优先级第 2 小的“Insert 1”（[第 16 行](#)）。

“Insert 1”正常执行插入操作，由于完成一次插入后满足非空条件，会唤醒阻塞的“Remove 0”，进入就绪队列（[第 18 行](#)），之后正常执行直到结束（[第 17-25 行](#)）。

此时，就绪队列中有“Insert 0”和“Remove 0”，因为“Remove 0”优先级更小，所以先运行（[第 26 行](#)）。

“Remove 0”执行第 1 次删除操作时，主动放弃 CPU（[第 29-30 行](#)），转到“Insert 0”（[第 31 行](#)）。

“Insert 0”由于锁被“Remove 0”拥有，无法对链表操作，被阻塞（[第 33 行](#)），转到“Remove 0”（[第 34 行](#)）。

“Remove 0”正常执行删除操作，完成一次删除操作后，会释放锁，唤醒“Insert 0”（[第 36 行](#)），进入就绪队列，因为“Remove 0”没有放弃 CPU，所

以“Insert 0”还不会执行，“Remove 0”继续执行删除操作（[第 37-40 行](#)），直到链表为空，不满足非空条件，被阻塞（[第 41 行](#)），转到“Insert 0”（[第 42 行](#)）。

“Insert 0”在完成插入操作前，想要主动放弃 CPU（[第 44 行](#)），但是由于“Remove 0”还在被阻塞，且没有其他进程，所以还是“Insert 0”运行，在执行完第一次插入后，唤醒“Remove 0”进入就绪队列（[第 45 行](#)）。

后续操作分析，与上述类似不再赘述。

结论：

通过上述分析，可以看出对于链表的访问是互斥的，这是锁机制正确实现的结果。也可以看到当条件不满足时，线程会被阻塞；条件满足后会被唤醒进入就绪队列，且不会立即执行。以及基于优先级调度的正确性。

4.2 验证线程安全的表结构实现的正确性

4.2.1 验证单线程下的表结构各种操作的正确性

命令：./nachos -q 3 -d e

该测试运行的是 test3 函数，无参数。

运行结果：

```
1 $ ./nachos -q 3 -d e
2 main: insert 0x8f6a170 in 0.
3 main: insert 0x8f6a180 in 1.
4 main: insert 0x8f6a190 in 2.
5 main: insert failed because the tabel is full.
6 main: get 0x8f6a170 in 0.
7 main: get 0x8f6a180 in 1.
8 main: get 0x8f6a190 in 2.
9 main: release 0x8f6a170 in 0.
10 main: insert 0x8f6a1b0 in 0.
11 main: release 0x8f6a180 in 1.
12 main: insert 0x8f6a1c0 in 1.
13 main: release 0x8f6a190 in 2.
14 main: insert 0x8f6a1d0 in 2.
15 main: release 0x8f6a1b0 in 0.
16 main: release 0x8f6a1c0 in 1.
17 main: release 0x8f6a1d0 in 2.
18 main: get NULL in 0 because not allocated.
19 main: get NULL in 1 because not allocated.
20 main: get NULL in 2 because not allocated.
21 No threads ready or runnable, and no pending interrupts.
22 Assuming the program completed.
23 Machine halting!
24
25 Ticks: total 390, idle 0, system 390, user 0
26 Disk I/O: reads 0, writes 0
27 Console I/O: reads 0, writes 0
```

```
28 Paging: faults 0
29 Network I/O: packets received 0, sent 0
30
31 Cleaning up...
```

分析:

所有断言 **ASSERT** 都是真的, 符合预期结果。

先向表申请了三个表槽, 占满了表 ([第 2-4 行](#))。

在表已经满的情况下, 再向表申请表槽, 会申请失败, 返回的是 -1 ([第 5 行](#))。

向表获取各索引的内容, 可以看到和申请时, 放入的内容一样 ([第 6-8 行](#))。

对于表的各位置释放, 再重新分配, 结果符合预期 ([第 9-14 行](#))。

释放表的所有内容, 并用 **Get** 函数查看是否成功释放, 获得到的都是 **NULL**, 所以成功释放 ([第 15-20 行](#))。

结论:

在单线程环境下, 各种情况下的表操作都是正确的。

4.2.3 验证多线程下的表结构各种操作的正确性

命令: `./nachos -q 4 size n1 n2 n3 -d e`

该测试运行的是 **test4** 函数, 有 4 个参数:

- **size** -- 表的大小
- **n1** -- **Alloc** 进程的数量
- **n2** -- **Get** 进程的数量
- **n3** -- **Release** 进程的数量。

【注】这三种进程都是执行对应操作一次, 且 **Get** 和 **Release** 都是在随机位置操作。

示例:

```
1 $ ./nachos -q 4 3 5 5 5 -d e
2 main: make thread Alloc 0 50
3 main: make thread Alloc 1 62
4 main: make thread Alloc 2 38
5 main: make thread Alloc 3 75
6 main: make thread Alloc 4 71
7 main: make thread Get 0 83
8 main: make thread Get 1 16
9 main: make thread Get 2 6
10 main: make thread Get 3 97
11 main: make thread Get 4 53
12 main: make thread Release 0 4
13 main: make thread Release 1 55
14 main: make thread Release 2 83
15 main: make thread Release 3 89
16 main: make thread Release 4 61
17 Release 0: release (nil) in 0.
18 Get 2: get NULL in 0 because not allocated.
19 Get 1: get NULL in 2 because not allocated.
```

```

20  Alloc 2: insert 0x8928300 in 0.
21  Alloc 0: insert 0x891c1f0 in 1.
22  Get 4: get 0x891c1f0 in 1.
23  Release 1: release 0x8928300 in 0.
24  Release 4: release (nil) in 2.
25  Alloc 1: insert 0x8922278 in 0.
26  Alloc 4: insert 0x8934410 in 2.
27  Alloc 3: insert failed because the tabel is full.
28  Get 0: get 0x8922278 in 0.
29  Release 2: release 0x891c1f0 in 1.
30  Release 3: release (nil) in 1.
31  Get 3: get 0x8922278 in 0.
32  No threads ready or runnable, and no pending interrupts.
33  Assuming the program completed.
34  Machine halting!
35
36  Ticks: total 610, idle 0, system 610, user 0
37  Disk I/O: reads 0, writes 0
38  Console I/O: reads 0, writes 0
39  Paging: faults 0
40  Network I/O: packets received 0, sent 0
41
42  Cleaning up...

```

分析:

上面的运行结果，可以总结为下图，可以看到即使在多线程环境下，Table 的三个操作也是正确的。

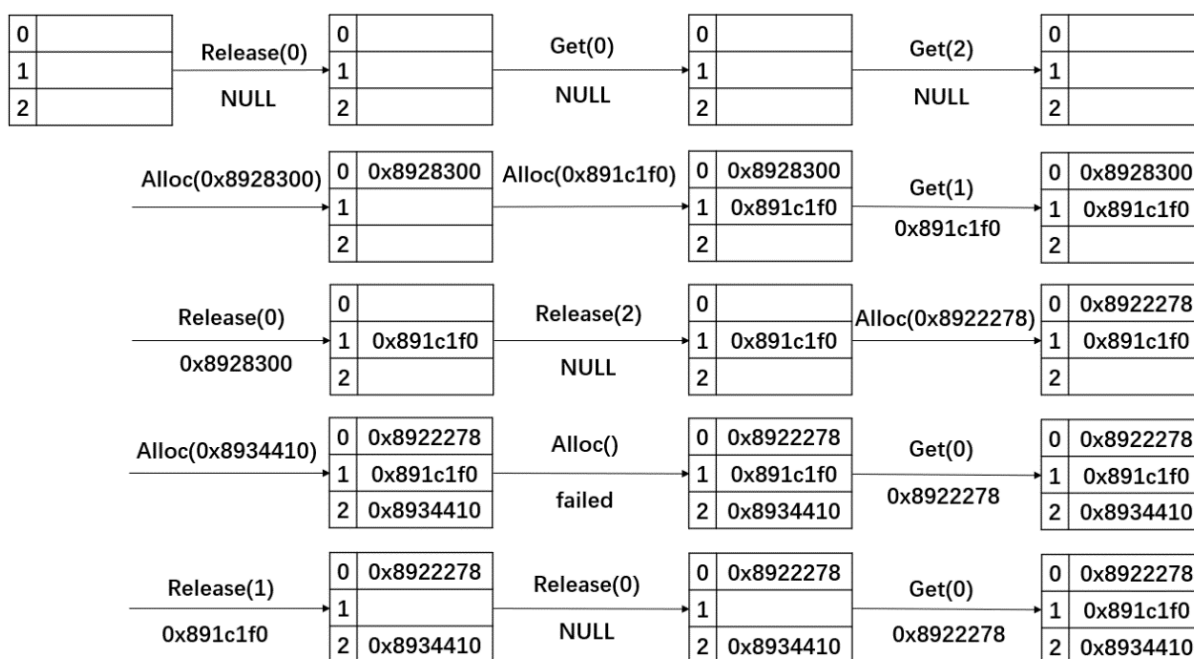


图 4.2.3-1

结论:

在尝试多种参数设置（十种以上）后，使结论具有一般性，在这些运行中，各线程的操作行为都符合预期，这验证了多线程下的表结构实现的正确性。

4.3 验证大小受限的缓冲区实现的正确性

命令: `./nachos -q 5 size t1 n1 t2 n2 -d e`

该测试运行的是 `test5` 函数，有 5 个参数:

- `Size` -- 缓冲区的大小，单位是字节
- `t1` -- `Writer` 进程的数量
- `n1` -- 每个 `Writer` 进程写入的字节数
- `t2` -- `Reader` 进程的数量
- `n2` -- 每个 `Reader` 进程读取的字节数。

【注】写入的数据是随机的小写英文字符串。

示例:

```
1 $ ./nachos -q 5 1 3 2 2 3 -d et -rs 1000
2 Forking thread "Writer 0" with func = 0x804ae8a, arg = 0
3 Putting thread Writer 0 on ready list.
4 main: make thread Writer 0 20
5 Forking thread "Writer 1" with func = 0x804ae8a, arg = 1
6 Putting thread Writer 1 on ready list.
7 main: make thread Writer 1 12
8 Forking thread "Writer 2" with func = 0x804ae8a, arg = 2
9 Putting thread Writer 2 on ready list.
10 main: make thread Reader 2 21
11 Forking thread "Reader 0" with func = 0x804ac98, arg = 0
12 Putting thread Reader 0 on ready list.
13 main: make thread Reader 0 50
14 Forking thread "Reader 1" with func = 0x804ac98, arg = 1
15 Putting thread Reader 1 on ready list.
16 main: make thread Reader 1 39
17 Finishing thread "main"
18 Sleeping thread "main"
19 Switching from thread "main" to thread "Writer 1"
20 Writer 1: want to write ug.
21 Writer 1: write u in 0.
22 Sleeping thread "Writer 1"
23 Switching from thread "Writer 1" to thread "Writer 0"
24 Writer 0: want to write gp.
25 Sleeping thread "Writer 0"
26 Switching from thread "Writer 0" to thread "Writer 2"
27 Writer 2: want to write pg.
28 Sleeping thread "Writer 2"
29 Switching from thread "Writer 2" to thread "Reader 1"
30 Reader 1: read u in 0.
```



```
31  Putting thread Writer 1 on ready list.
32  Sleeping thread "Reader 1"
33  Switching from thread "Reader 1" to thread "Writer 1"
34  Now in thread "Writer 1"
35  Deleting thread "main"
36  Writer 1: write g in 0.
37  Putting thread Reader 1 on ready list.
38  Putting thread Writer 0 on ready list.
39  Finishing thread "Writer 1"
40  Sleeping thread "Writer 1"
41  Switching from thread "Writer 1" to thread "Writer 0"
42  Now in thread "Writer 0"
43  Deleting thread "Writer 1"
44  Yielding thread "Writer 0"
45  Putting thread Writer 0 on ready list.
46  Switching from thread "Writer 0" to thread "Reader 1"
47  Now in thread "Reader 1"
48  Reader 1: read g in 0.
49  Sleeping thread "Reader 1"
50  Switching from thread "Reader 1" to thread "Writer 0"
51  Now in thread "Writer 0"
52  Writer 0: write g in 0.
53  Putting thread Reader 1 on ready list.
54  Sleeping thread "Writer 0"
55  Switching from thread "Writer 0" to thread "Reader 1"
56  Now in thread "Reader 1"
57  Reader 1: read g in 0.
58  Putting thread Writer 0 on ready list.
59  Yielding thread "Reader 1"
60  Putting thread Reader 1 on ready list.
61  Switching from thread "Reader 1" to thread "Writer 0"
62  Now in thread "Writer 0"
63  Writer 0: write p in 0.
64  Putting thread Writer 2 on ready list.
65  Finishing thread "Writer 0"
66  Sleeping thread "Writer 0"
67  Switching from thread "Writer 0" to thread "Writer 2"
68  Now in thread "Writer 2"
69  Deleting thread "Writer 0"
70  Sleeping thread "Writer 2"
71  Switching from thread "Writer 2" to thread "Reader 1"
72  Now in thread "Reader 1"
```

```
73 Reader 1: read ugg.
74 Finishing thread "Reader 1"
75 Sleeping thread "Reader 1"
76 Switching from thread "Reader 1" to thread "Reader 0"
77 Yielding thread "Reader 0"
78 Reader 0: read p in 0.
79 Putting thread Writer 2 on ready list.
80 Sleeping thread "Reader 0"
81 Switching from thread "Reader 0" to thread "Writer 2"
82 Now in thread "Writer 2"
83 Deleting thread "Reader 1"
84 Writer 2: write p in 0.
85 Putting thread Reader 0 on ready list.
86 Sleeping thread "Writer 2"
87 Switching from thread "Writer 2" to thread "Reader 0"
88 Now in thread "Reader 0"
89 Reader 0: read p in 0.
90 Putting thread Writer 2 on ready list.
91 Sleeping thread "Reader 0"
92 Switching from thread "Reader 0" to thread "Writer 2"
93 Now in thread "Writer 2"
94 Writer 2: write g in 0.
95 Putting thread Reader 0 on ready list.
96 Finishing thread "Writer 2"
97 Sleeping thread "Writer 2"
98 Switching from thread "Writer 2" to thread "Reader 0"
99 Now in thread "Reader 0"
100 Deleting thread "Writer 2"
101 Reader 0: read g in 0.
102 Yielding thread "Reader 0"
103 Reader 0: read ppg.
104 Finishing thread "Reader 0"
105 Sleeping thread "Reader 0"
106 No threads ready or runnable, and no pending interrupts.
107 Assuming the program completed.
108 Machine halting!
109
110 Ticks: total 602, idle 112, system 490, user 0
111 Disk I/O: reads 0, writes 0
112 Console I/O: reads 0, writes 0
113 Paging: faults 0
114 Network I/O: packets received 0, sent 0
```

115

116 Cleaning up...

分析:

主线程创建了五个进程，其中三个是生产者进程分别是“Writer 0”、“Writer 1”、“Writer 2”，另外两个是消费者进程分别是“Reader 0”、“Reader 1”（[第 2-16 行](#)）。

有如下的结果:

	Writer 0	Writer 1	Writer 2	Reader 0	Reader 1
p	20	12	21	50	39
data	gp	ug	pg	ppg	ugg

从结果上，生产者进程按顺序的生产数据，而不会交叉生产，即便发生了生产者进程之间的切换，比如说“Writer 1”在写入 u 后（[第 21 行](#)），再次尝试写入因为缓冲区已满，而被阻塞（[第 22 行](#)），转到“Writer 0”（[第 23 行](#)），因为“Writer 1”还没完全写完，未释放 writeLock，所以“Writer 0”无法获得锁，被阻塞（[第 25 行](#)），“Writer 2”同理（[第 28 行](#)）。

结论:

在尝试多种参数设置（十种以上）后，使结论具有一般性，在这些运行中，各线程的操作行为都符合预期，这验证了大小受限的缓冲区实现的正确性。

4.4 验证优先级调度程序的正确性

命令: `./nachos -q 6 n -d e`

该测试运行的是 test6 函数，有一个参数:

- n -- 创建的进程的数量

运行结果:

```
1 $ ./nachos -q 6 10 -d e
2 main: make thread mythread 0 83
3 main: make thread mythread 1 86
4 main: make thread mythread 2 77
5 main: make thread mythread 3 15
6 main: make thread mythread 4 93
7 main: make thread mythread 5 35
8 main: make thread mythread 6 86
9 main: make thread mythread 7 92
10 main: make thread mythread 8 49
11 main: make thread mythread 9 21
12 mythread 3 15 begin to run.
13 mythread 9 21 begin to run.
14 mythread 5 35 begin to run.
15 mythread 8 49 begin to run.
16 mythread 2 77 begin to run.
17 mythread 0 83 begin to run.
18 mythread 1 86 begin to run.
19 mythread 6 86 begin to run.
20 mythread 7 92 begin to run.
```

```

21 mythread 4 93 begin to run.
22 No threads ready or runnable, and no pending interrupts.
23 Assuming the program completed.
24 Machine halting!
25
26 Ticks: total 210, idle 0, system 210, user 0
27 Disk I/O: reads 0, writes 0
28 Console I/O: reads 0, writes 0
29 Paging: faults 0
30 Network I/O: packets received 0, sent 0
31
32 Cleaning up...

```

分析:

从上面的运行结果可以看出,线程是按照优先级顺序调度的,而当优先级相同时,比如“mythread 1”和“mythread 6”优先级相同都是 86,它们的调用顺序是按照创建顺序,即“mythread 1”先于“mythread 6”调度([第 18-19 行](#))。所以基于优先级的调度程序是正确的。

4.5 利用优先级调度程序观察不同优先级的生产者消费者问题

命令: `./nachos -q size p0 n0 p1 n1 p2 n2 -d e -y 100`

该测试运行的是 test7 函数,函数有 7 个参数:

- size -- 缓冲区的大小
- p0 -- “Writer 0” 的优先级
- n0 -- “Writer 0” 调用 Write 的次数
- p1 -- “Writer 1” 的优先级
- n1 -- “Writer 1” 调用 Write 的次数
- p2 -- “Reader 0” 的优先级
- n2 -- “Reader 0” 调用 Read 的次数

【注】为了简便且不失一般性,Write 或 Read 函数都只对两个字节操作。

通过运行不同的参数,观察生产者和消费者的行为,可以总结为如下的表格,为了简便,“Writer 0”依次调用三次 Write 函数,分别写入 ab、cd、ef;“Writer 1”调用两次 Write 函数,分别写入 AB、CD;“Reader 0”调用五次 Read 函数;优先级分为小中大,三个层次,优先级小的先运行,可以分别记为 1、2、3;切换(`currentThread->Yield()`)的位置选在每次调用完 Read 或 Write 函数。为了避免意外的线程切换,将缓冲区的大小设置为足够容纳所有 Write 的字节。分为启用切换和不启用切换,记录所有优先级情况下,“Reader 0”的读取结果。

Yield	Writer 0	Writer 1	p0	p1	p2	Reader 0
No	ab cd ef	AB CD	1	2	3	ab cd ef AB CD
No	ab cd ef	AB CD	1	3	2	ab cd ef AB CD
No	ab cd ef	AB CD	2	1	3	AB CD ab cd ef
No	ab cd ef	AB CD	2	3	1	ab cd ef AB CD
No	ab cd ef	AB CD	3	1	2	AB CD ab cd ef
No	ab cd ef	AB CD	3	2	1	AB CD ab cd ef

Yes	ab cd ef	AB CD	1	2	3	ab AB cd CD ef
Yes	ab cd ef	AB CD	1	3	2	ab cd ef AB CD
Yes	ab cd ef	AB CD	2	1	3	AB ab CD cd ef
Yes	ab cd ef	AB CD	2	3	1	ab cd ef AB CD
Yes	ab cd ef	AB CD	3	1	2	AB CD ab cd ef
Yes	ab cd ef	AB CD	3	2	1	AB CD ab cd ef

结论：

- a. 只有当消费者的优先级最低时，有无 **Yield** 对于生产者之间的行为产生影响，这是因为在没有 **Yield** 时，两个生产者按照优先级顺序生产，而有 **Yield** 时，会在生产一次之后，就发生线程的切换，使得两个生产者交替生产。由于在本测试下不会发生阻塞，所以线程切换是发生在两个优先级最高的线程之间的，所以当消费者非最低优先级时，消费者消费的内容无差别，只是影响到消费者消费的时机，有 **Yield** 时，消费者可以更早的开始消费。

五、实验总结

5.1 信号量实现条件变量错误

刚开始用信号量实现条件变量的时候，错误地认为条件变量和锁类似都用初始值为 **1** 的信号量模拟，但这是错误的，因为当条件不满足时，**Wait** 函数调用 **P** 操作不会被阻塞，这是不符合预期的。正确的是使用初始值为 **0** 的信号量，并通过记录阻塞线程的数量，判断是否需要 **V** 操作。

5.2 条件变量的 **Wait** 函数中，“**waitNum++**”位置错误

在改完条件变量信号量的初始值后，又出现了错误，通过 **DEBUG** 发现是 **Wait** 函数中，**P** 操作和 “**waitNum++**” 操作顺序错误，因为先 **P** 操作被阻塞，的话，由于 **waitNum** 的数量还没有改变，没有线程能通过 **Signal** 唤醒被阻塞的线程，所以应该先递增再阻塞。

5.3 **Write** 和 **Read** 函数实现错误

由于没有认真读题，错误的认为 **Write** 函数之间可以交叉的写入数据，所以只使用锁保护一个字节的写入操作，在每次写入一个字节释放锁后，发生线程切换就可能会出现数据交叉。所以最终实现是对整个 **Write** 或 **Read** 函数上锁。

六、实验心得

在本次实验中，掌握了锁机制和条件变量实现的两种方法，在深入了解锁机制和条件变量的基础上，正确实现了锁机制和条件变量，并了解了如何设计线程安全的程序，即对于临界资源的访问，需要通过锁来保证互斥性。通过本次实验也了解到了许多编程上的技巧，比如如何正确使用命令行参数，如何减少代码之间的冲突，以及如何 **DEBUG**。还有通过仔细查阅线程调度的相关文件，了解到了线程调度的实现，以及线程是如何在各种状态之间转变的。

七、附件

附件 1 **synch-sleep.h**

```

1 // synch.h
2 // Data structures for synchronizing threads.
3 //
4 // Three kinds of synchronization are defined here: semaphores,
```

```

5 // locks, and condition variables. The implementation for
6 // semaphores is given; for the latter two, only the procedure
7 // interface is given -- they are to be implemented as part of
8 // the first assignment.
9 //
10 // Note that all the synchronization objects take a "name" as
11 // part of the initialization. This is solely for debugging purposes.
12 //
13 // Copyright (c) 1992-1993 The Regents of the University of California.
14 // All rights reserved. See copyright.h for copyright notice and
15 // limitation
16 // synch.h -- synchronization primitives.
17
18 #ifndef SYNCH_H
19 #define SYNCH_H
20
21 #include "copyright.h"
22 #include "thread.h"
23 #include "list.h"
24
25 // The following class defines a "semaphore" whose value is a non-negative
26 // integer. The semaphore has only two operations P() and V():
27 //
28 // P() -- waits until value > 0, then decrement
29 //
30 // V() -- increment, waking up a thread waiting in P() if necessary
31 //
32 // Note that the interface does not allow a thread to read the value of
33 // the semaphore directly -- even if you did read the value, the
34 // only thing you would know is what the value used to be. You don't
35 // know what the value is now, because by the time you get the value
36 // into a register, a context switch might have occurred,
37 // and some other thread might have called P or V, so the true value might
38 // now be different.
39
40 class Semaphore {
41 public:
42     Semaphore(char* debugName, int initialValue); // set initial value
43     ~Semaphore(); // de-allocate semaphore
44     char* getName() { return name; } // debugging assist
45
46     void P(); // these are the only operations on a semaphore

```

```

47     void V(); // they are both *atomic*
48
49 private:
50     char* name;        // useful for debugging
51     int value;         // semaphore value, always >= 0
52     List *queue;       // threads waiting in P() for the value to be > 0
53 };
54
55 // The following class defines a "lock". A lock can be BUSY or FREE.
56 // There are only two operations allowed on a lock:
57 //
58 // Acquire -- wait until the lock is FREE, then set it to BUSY
59 //
60 // Release -- set lock to be FREE, waking up a thread waiting
61 //           in Acquire if necessary
62 //
63 // In addition, by convention, only the thread that acquired the lock
64 // may release it. As with semaphores, you can't read the lock value
65 // (because the value might change immediately after you read it).
66
67 class Lock {
68 public:
69     Lock(char* debugName); // initialize lock to be FREE
70     ~Lock();               // deallocate lock
71     char* getName() { return name; } // debugging assist
72
73     void Acquire(); // these are the only operations on a lock
74     void Release(); // they are both *atomic*
75
76     bool isHeldByCurrentThread(); // true if the current thread
77     // holds this lock. Useful for
78     // checking in Release, and in
79     // Condition variable ops below.
80
81 private:
82     char* name; // for debugging
83     // plus some other stuff you'll need to define
84     List *queue;
85     Thread *lockOwner;
86 };
87
88 // The following class defines a "condition variable". A condition

```

```

89 // variable does not have a value, but threads may be queued, waiting
90 // on the variable. These are only operations on a condition variable:
91 //
92 // Wait() -- release the lock, relinquish the CPU until signaled,
93 // then re-acquire the lock
94 //
95 // Signal() -- wake up a thread, if there are any waiting on
96 // the condition
97 //
98 // Broadcast() -- wake up all threads waiting on the condition
99 //
100 // All operations on a condition variable must be made while
101 // the current thread has acquired a lock. Indeed, all accesses
102 // to a given condition variable must be protected by the same lock.
103 // In other words, mutual exclusion must be enforced among threads calling
104 // the condition variable operations.
105 //
106 // In Nachos, condition variables are assumed to obey *Mesa*-style
107 // semantics. When a Signal or Broadcast wakes up another thread,
108 // it simply puts the thread on the ready list, and it is the
109 responsibility
110 // of the woken thread to re-acquire the lock (this re-acquire is
111 // taken care of within Wait()). By contrast, some define condition
112 // variables according to *Hoare*-style semantics -- where the signalling
113 // thread gives up control over the lock and the CPU to the woken thread,
114 // which runs immediately and gives back control over the lock to the
115 // signaller when the woken thread leaves the critical section.
116 //
117 // The consequence of using Mesa-style semantics is that some other thread
118 // can acquire the lock, and change data structures, before the woken
119 // thread gets a chance to run.
120
121 class Condition {
122 public:
123     Condition(char* debugName); // initialize condition to
124         // "no one waiting"
125     ~Condition(); // deallocate the condition
126     char* getName() { return (name); }
127
128     void Wait(Lock *conditionLock); // these are the 3 operations on
129         // condition variables; releasing the
130         // lock and going to sleep are

```



```

131         // *atomic* in Wait()
132     void Signal(Lock *conditionLock); // conditionLock must be held by
133     void Broadcast(Lock *conditionLock); // the currentThread for all of
134         // these operations
135
136     private:
137         char* name;
138         // plus some other stuff you'll need to define
139         List *queue;
140 };
141 #endif // SYNCH_H

```

附件 2 synch-sleep.cc

```

1 // synch.cc
2 // Routines for synchronizing threads. Three kinds of
3 // synchronization routines are defined here: semaphores, locks
4 // and condition variables (the implementation of the last two
5 // are left to the reader).
6 //
7 // Any implementation of a synchronization routine needs some
8 // primitive atomic operation. We assume Nachos is running on
9 // a uniprocessor, and thus atomicity can be provided by
10 // turning off interrupts. While interrupts are disabled, no
11 // context switch can occur, and thus the current thread is guaranteed
12 // to hold the CPU throughout, until interrupts are reenabled.
13 //
14 // Because some of these routines might be called with interrupts
15 // already disabled (Semaphore::V for one), instead of turning
16 // on interrupts at the end of the atomic operation, we always simply
17 // re-set the interrupt state back to its original value (whether
18 // that be disabled or enabled).
19 //
20 // Copyright (c) 1992-1993 The Regents of the University of California.
21 // All rights reserved. See copyright.h for copyright notice and
22 // limitation
23 // of liability and disclaimer of warranty provisions.
24
25 #include "copyright.h"
26 #include "synch-sleep.h"
27 #include "system.h"
28
29 //-----
30 // Semaphore::Semaphore
31 // Initialize a semaphore, so that it can be used for synchronization.

```

```

32 //
33 // "debugName" is an arbitrary name, useful for debugging.
34 // "initialValue" is the initial value of the semaphore.
35 //-----
36
37 Semaphore::Semaphore(char *debugName, int initialValue)
38 {
39     name = debugName;
40     value = initialValue;
41     queue = new List;
42 }
43
44 //-----
45 // Semaphore::Semaphore
46 // De-allocate semaphore, when no longer needed. Assume no one
47 // is still waiting on the semaphore!
48 //-----
49
50 Semaphore::~Semaphore()
51 {
52     delete queue;
53 }
54
55 //-----
56 // Semaphore::P
57 // Wait until semaphore value > 0, then decrement. Checking the
58 // value and decrementing must be done atomically, so we
59 // need to disable interrupts before checking the value.
60 //
61 // Note that Thread::Sleep assumes that interrupts are disabled
62 // when it is called.
63 //-----
64
65 void Semaphore::P()
66 {
67     IntStatus oldLevel = interrupt->SetLevel(IntOff); // disable interrupts
68
69     while (value == 0)
70     {
71         // semaphore not available
72         queue->Append((void *)currentThread); // so go to sleep
73         currentThread->Sleep();
74     }

```

```

74     value--; // semaphore available,
75             // consume its value
76
77     (void)interrupt->SetLevel(oldLevel); // re-enable interrupts
78 }
79
80 //-----
81 // Semaphore::V
82 // Increment semaphore value, waking up a waiter if necessary.
83 // As with P(), this operation must be atomic, so we need to disable
84 // interrupts. Scheduler::ReadyToRun() assumes that threads
85 // are disabled when it is called.
86 //-----
87
88 void Semaphore::V()
89 {
90     Thread *thread;
91     IntStatus oldLevel = interrupt->SetLevel(IntOff);
92
93     thread = (Thread *)queue->Remove();
94     if (thread != NULL) // make thread ready, consuming the V immediately
95         scheduler->ReadyToRun(thread);
96     value++;
97     (void)interrupt->SetLevel(oldLevel);
98 }
99
100 // Dummy functions -- so we can compile our later assignments
101 // Note -- without a correct implementation of Condition::Wait(),
102 // the test case in the network assignment won't work!
103 Lock::Lock(char *debugName)
104 {
105     name = debugName;
106     queue = new List;
107     lockOwner = NULL;
108 }
109 Lock::~~Lock()
110 {
111     delete queue;
112 }
113 void Lock::Acquire()
114 {
115     IntStatus oldLevel = interrupt->SetLevel(IntOff); // 中断禁用

```

```

116
117     ASSERT(!isHeldByCurrentThread()) // 断言当前线程未拥锁
118
119     while (lockOwner != NULL)
120     {
121         queue->Append((void *) currentThread);
122         currentThread->Sleep();
123     }
124     lockOwner = currentThread;
125
126     (void)interrupt->SetLevel(oldLevel); // 恢复中断标志状态
127 }
128 void Lock::Release()
129 {
130     IntStatus oldLevel = interrupt->SetLevel(IntOff); // 中断禁用
131
132     ASSERT(isHeldByCurrentThread()) // 断言当前线程释放的是自己获得的锁
133
134     Thread *thread = (Thread *)queue->Remove();
135     if (thread != NULL) scheduler->ReadyToRun(thread);
136     if (currentThread == lockOwner) lockOwner = NULL;
137
138     (void)interrupt->SetLevel(oldLevel); // 恢复中断标志状态
139 }
140 bool Lock::isHeldByCurrentThread()
141 {
142     return lockOwner == currentThread;
143 }
144
145 Condition::Condition(char *debugName)
146 {
147     name = debugName;
148     queue = new List;
149 }
150 Condition::~Condition()
151 {
152     delete queue;
153 }
154 void Condition::Wait(Lock *conditionLock)
155 {
156     IntStatus oldLevel = interrupt->SetLevel(IntOff); // 中断禁用
157

```

```

158     // ASSERT(FALSE);
159     ASSERT(conditionLock->isHeldByCurrentThread());
160
161     conditionLock->Release();    // 释放锁
162     queue->Append((void *) currentThread);
163     currentThread->Sleep();      // 放弃 CPU
164
165     conditionLock->Acquire();    // 请求锁
166
167     (void)interrupt->SetLevel(oldLevel);    // 恢复中断标志状态
168 }
169 void Condition::Signal(Lock *conditionLock)
170 {
171     IntStatus oldLevel = interrupt->SetLevel(IntOff);    // 中断禁用
172
173     ASSERT(conditionLock->isHeldByCurrentThread());
174
175     Thread *thread = (Thread *)queue->Remove();
176     if (thread != NULL) scheduler->ReadyToRun(thread);
177
178     (void)interrupt->SetLevel(oldLevel);    // 恢复中断标志状态
179 }
180 void Condition::Broadcast(Lock *conditionLock)
181 {
182     IntStatus oldLevel = interrupt->SetLevel(IntOff);    // 中断禁用
183
184     ASSERT(conditionLock->isHeldByCurrentThread());
185
186     Thread *thread = (Thread *)queue->Remove();
187     while (thread != NULL)
188     {
189         scheduler->ReadyToRun(thread);
190         thread = (Thread *)queue->Remove();
191     }
192
193     (void)interrupt->SetLevel(oldLevel);    // 恢复中断标志状态
194 }

```

附件 3 synch-sem.h

```

1 // synch.h
2 // Data structures for synchronizing threads.
3 //
4 // Three kinds of synchronization are defined here: semaphores,
5 // locks, and condition variables. The implementation for

```

```

6 // semaphores is given; for the latter two, only the procedure
7 // interface is given -- they are to be implemented as part of
8 // the first assignment.
9 //
10 // Note that all the synchronization objects take a "name" as
11 // part of the initialization. This is solely for debugging purposes.
12 //
13 // Copyright (c) 1992-1993 The Regents of the University of California.
14 // All rights reserved. See copyright.h for copyright notice and
15 // limitation
16 // synch.h -- synchronization primitives.
17
18 #ifndef SYNCH_H
19 #define SYNCH_H
20
21 #include "copyright.h"
22 #include "thread.h"
23 #include "list.h"
24
25 // The following class defines a "semaphore" whose value is a non-negative
26 // integer. The semaphore has only two operations P() and V():
27 //
28 // P() -- waits until value > 0, then decrement
29 //
30 // V() -- increment, waking up a thread waiting in P() if necessary
31 //
32 // Note that the interface does not allow a thread to read the value of
33 // the semaphore directly -- even if you did read the value, the
34 // only thing you would know is what the value used to be. You don't
35 // know what the value is now, because by the time you get the value
36 // into a register, a context switch might have occurred,
37 // and some other thread might have called P or V, so the true value might
38 // now be different.
39
40 class Semaphore {
41 public:
42     Semaphore(char* debugName, int initialValue); // set initial value
43     ~Semaphore(); // de-allocate semaphore
44     char* getName() { return name; } // debugging assist
45
46     void P(); // these are the only operations on a semaphore
47     void V(); // they are both atomic

```

```

48
49     private:
50         char* name;           // useful for debugging
51         int value;            // semaphore value, always >= 0
52         List *queue;          // threads waiting in P() for the value to be > 0
53     };
54
55     // The following class defines a "lock". A lock can be BUSY or FREE.
56     // There are only two operations allowed on a lock:
57     //
58     // Acquire -- wait until the lock is FREE, then set it to BUSY
59     //
60     // Release -- set lock to be FREE, waking up a thread waiting
61     //           in Acquire if necessary
62     //
63     // In addition, by convention, only the thread that acquired the lock
64     // may release it. As with semaphores, you can't read the lock value
65     // (because the value might change immediately after you read it).
66
67     class Lock {
68     public:
69         Lock(char* debugName);    // initialize lock to be FREE
70         ~Lock();                 // deallocate lock
71         char* getName() { return name; } // debugging assist
72
73         void Acquire(); // these are the only operations on a lock
74         void Release(); // they are both *atomic*
75
76         bool isHeldByCurrentThread(); // true if the current thread
77             // holds this lock. Useful for
78             // checking in Release, and in
79             // Condition variable ops below.
80     private:
81         char* name;             // for debugging
82         // plus some other stuff you'll need to define
83         Semaphore *lock;
84         Thread *lockOwner;
85     };
86
87     // The following class defines a "condition variable". A condition
88     // variable does not have a value, but threads may be queued, waiting
89     // on the variable. These are only operations on a condition variable:

```

```

90 //
91 // Wait() -- release the lock, relinquish the CPU until signaled,
92 //    then re-acquire the lock
93 //
94 // Signal() -- wake up a thread, if there are any waiting on
95 //    the condition
96 //
97 // Broadcast() -- wake up all threads waiting on the condition
98 //
99 // All operations on a condition variable must be made while
100 // the current thread has acquired a lock. Indeed, all accesses
101 // to a given condition variable must be protected by the same lock.
102 // In other words, mutual exclusion must be enforced among threads calling
103 // the condition variable operations.
104 //
105 // In Nachos, condition variables are assumed to obey *Mesa*-style
106 // semantics. When a Signal or Broadcast wakes up another thread,
107 // it simply puts the thread on the ready list, and it is the
108 responsibility
109 // of the woken thread to re-acquire the lock (this re-acquire is
110 // taken care of within Wait()). By contrast, some define condition
111 // variables according to *Hoare*-style semantics -- where the signalling
112 // thread gives up control over the lock and the CPU to the woken thread,
113 // which runs immediately and gives back control over the lock to the
114 // signaller when the woken thread leaves the critical section.
115 //
116 // The consequence of using Mesa-style semantics is that some other thread
117 // can acquire the lock, and change data structures, before the woken
118 // thread gets a chance to run.
119
120 class Condition {
121 public:
122     Condition(char* debugName); // initialize condition to
123         // "no one waiting"
124     ~Condition(); // deallocate the condition
125     char* getName() { return (name); }
126     void Wait(Lock *conditionLock); // these are the 3 operations on
127         // condition variables; releasing the
128         // lock and going to sleep are
129         // *atomic* in Wait()
130     void Signal(Lock *conditionLock); // conditionLock must be held by
131     void Broadcast(Lock *conditionLock); // the currentThread for all of

```



```

132         // these operations
133     private:
134         char* name;
135         // plus some other stuff you'll need to define
136         Semaphore *condition;
137         int waitNum;
138 };
139 #endif // SYNCH_H

```

附件 4 synch-sem.cc

```

1 // synch.cc
2 // Routines for synchronizing threads. Three kinds of
3 // synchronization routines are defined here: semaphores, locks
4 // and condition variables (the implementation of the last two
5 // are left to the reader).
6 //
7 // Any implementation of a synchronization routine needs some
8 // primitive atomic operation. We assume Nachos is running on
9 // a uniprocessor, and thus atomicity can be provided by
10 // turning off interrupts. While interrupts are disabled, no
11 // context switch can occur, and thus the current thread is guaranteed
12 // to hold the CPU throughout, until interrupts are reenabled.
13 //
14 // Because some of these routines might be called with interrupts
15 // already disabled (Semaphore::V for one), instead of turning
16 // on interrupts at the end of the atomic operation, we always simply
17 // re-set the interrupt state back to its original value (whether
18 // that be disabled or enabled).
19 //
20 // Copyright (c) 1992-1993 The Regents of the University of California.
21 // All rights reserved. See copyright.h for copyright notice and
22 // limitation
23 // of liability and disclaimer of warranty provisions.
24
25 #include "copyright.h"
26 #include "synch-sem.h"
27 #include "system.h"
28
29 //-----
30 // Semaphore::Semaphore
31 // Initialize a semaphore, so that it can be used for synchronization.
32 //
33 // "debugName" is an arbitrary name, useful for debugging.
34 // "initialValue" is the initial value of the semaphore.

```

```

35 //-----
36
37 Semaphore::Semaphore(char *debugName, int initialValue)
38 {
39     name = debugName;
40     value = initialValue;
41     queue = new List;
42 }
43
44 //-----
45 // Semaphore::Semaphore
46 // De-allocate semaphore, when no longer needed. Assume no one
47 // is still waiting on the semaphore!
48 //-----
49
50 Semaphore::~Semaphore()
51 {
52     delete queue;
53 }
54
55 //-----
56 // Semaphore::P
57 // Wait until semaphore value > 0, then decrement. Checking the
58 // value and decrementing must be done atomically, so we
59 // need to disable interrupts before checking the value.
60 //
61 // Note that Thread::Sleep assumes that interrupts are disabled
62 // when it is called.
63 //-----
64
65 void Semaphore::P()
66 {
67     IntStatus oldLevel = interrupt->SetLevel(IntOff); // disable interrupts
68
69     while (value == 0)
70     {
71         // semaphore not available
72         queue->Append((void *)currentThread); // so go to sleep
73         currentThread->Sleep();
74     }
75     value--; // semaphore available,
76             // consume its value

```

```

77     (void)interrupt->SetLevel(oldLevel); // re-enable interrupts
78 }
79
80 //-----
81 // Semaphore::V
82 // Increment semaphore value, waking up a waiter if necessary.
83 // As with P(), this operation must be atomic, so we need to disable
84 // interrupts. Scheduler::ReadyToRun() assumes that threads
85 // are disabled when it is called.
86 //-----
87
88 void Semaphore::V()
89 {
90     Thread *thread;
91     IntStatus oldLevel = interrupt->SetLevel(IntOff);
92
93     thread = (Thread *)queue->Remove();
94     if (thread != NULL) // make thread ready, consuming the V immediately
95         scheduler->ReadyToRun(thread);
96     value++;
97     (void)interrupt->SetLevel(oldLevel);
98 }
99
100 // Dummy functions -- so we can compile our later assignments
101 // Note -- without a correct implementation of Condition::Wait(),
102 // the test case in the network assignment won't work!
103 Lock::Lock(char *debugName)
104 {
105     name = debugName;
106     lock = new Semaphore("LockSemaphore", 1);
107     lockOwner = NULL;
108 }
109 Lock::~~Lock()
110 {
111     delete lock;
112 }
113 void Lock::Acquire()
114 {
115     ASSERT(!isHeldByCurrentThread());
116     lock->P();
117     lockOwner = currentThread;
118 }

```

```
119 void Lock::Release()
120 {
121     ASSERT(isHeldByCurrentThread());
122     if(currentThread == lockOwner)
123     {
124         lockOwner = NULL;
125         lock->V();
126     }
127 }
128 bool Lock::isHeldByCurrentThread()
129 {
130     return lockOwner == currentThread;
131 }
132
133 Condition::Condition(char *debugName)
134 {
135     name = debugName;
136     condition = new Semaphore("CondSemaphore", 0);
137     waitNum = 0;
138 }
139 Condition::~~Condition()
140 {
141     delete condition;
142 }
143 void Condition::Wait(Lock *conditionLock)
144 {
145     // ASSERT(FALSE);
146     conditionLock->Release();
147     ++ waitNum;
148     condition->P();
149     conditionLock->Acquire();
150 }
151 void Condition::Signal(Lock *conditionLock)
152 {
153     if(waitNum > 0)
154     {
155         -- waitNum;
156         condition->V();
157     }
158 }
159 void Condition::Broadcast(Lock *conditionLock)
160 {
```

```

161     while(waitNum > 0)
162     {
163         -- waitNum;
164         condition->V();
165     }

```

附件 5 dllist.h

```

#ifndef DLLIST_H
#define DLLIST_H
#include "synch.h"
class DLLElement
{
public:
    DLLElement(void *itemPtr, int sortKey); // initialize a list element
    DLLElement *next;                      // next element on list
                                           // NULL if this is the last
    DLLElement *prev;                     // previous element on list
                                           // NULL if this is the first
    int key;                               // priority, for a sorted list
    void *item;                            // pointer to item on the list
};
class DLList
{
public:
    DLList();                             // initialize the list
    ~DLList();                             // de-allocate the list
    void Prepend(void *item); // add to head of list (set key = min_key-1)
    void Append(void *item);  // add to tail of list (set key = max_key+1)
    void *Remove(int *keyPtr); // remove from head of list
                               // set *keyPtr to key of the removed item
                               // return item (or NULL if list is empty)
    bool IsEmpty();           // return true if list has elements
    // routines to put/get items on/off list in order (sorted by key)
    void SortedInsert(void *item, int sortKey);
    void *SortedRemove(int sortKey); // remove first item with key==sortKey
    // return NULL if no such item exists
    void show(); // show the element of list
private:
    DLLElement *first; // head of the list, NULL if empty
    DLLElement *last;  // last element of the list, NULL if empty
    Lock *lock;
    Condition *listEmpty;
};
#endif

```

附件 6 dllist.cc

```
1  #include <iostream>
2  #include "dllist.h"
3  #include "thread.h"
4  #include "synch.h"
5
6  #define KEY 50
7
8  extern Thread *currentThread;
9  extern void YIELD(int place);
10 extern void DEBUG(char flag, char *format, ...);
11
12 DLLElement::DLLElement(void * itemPtr, int sortKey)
13 {
14     YIELD(1);
15     next = NULL;
16     YIELD(2);
17     prev = NULL;
18     YIELD(3);
19     key = sortKey;
20     YIELD(4);
21     item = itemPtr;
22     YIELD(5);
23 }
24
25 DLLList::DLLList()
26 {
27     first = NULL;
28     last = NULL;
29     lock = new Lock("ListLock");
30     listEmpty = new Condition("ListEmptyCond");
31 }
32
33 DLLList::~~DLLList()
34 {
35     int key;
36     while(!IsEmpty())
37     {
38         Remove(&key);
39     }
40     delete lock;
41     delete listEmpty;
42 }
```

```
43
44 void DLLList::Prepend(void *item)
45 {
46     lock->Acquire();
47     if(IsEmpty())
48     {
49         DLLElement *elm = new DLLElement(item, KEY);
50         first = elm;
51         last = elm;
52     }
53     else
54     {
55         DLLElement *elm = new DLLElement(item, first->key - 1);
56         elm->next = first;
57         first->prev = elm;
58         first = elm;
59     }
60     listEmpty->Signal(lock);
61     lock->Release();
62 }
63
64 void DLLList::Append(void *item)
65 {
66     lock->Acquire();
67     if(IsEmpty())
68     {
69         DLLElement *elm = new DLLElement(item, KEY);
70         first = elm;
71         last = elm;
72     }
73     else
74     {
75         DLLElement *elm = new DLLElement(item, last->key + 1);
76         elm->prev = last;
77         last->next = elm;
78         last = elm;
79     }
80     listEmpty->Signal(lock);
81     lock->Release();
82 }
83
84 void *DLLList::Remove(int *keyPtr)
```

```

85 {
86     lock->Acquire();
87     while (IsEmpty()) listEmpty->Wait(lock);
88     YIELD(6);
89     if(IsEmpty())
90     {
91         YIELD(7);
92         *keyPtr = -1;
93         YIELD(8);
94         DEBUG('e', "[ERROR]%s : list is empty!\n",
95 currentThread->getName());
96         lock->Release();
97         return NULL;
98     }
99     else
100    {
101        YIELD(9);
102        void *item = first->item;
103        YIELD(10);
104        *keyPtr = first->key;
105        YIELD(11);
106        DLLElement *temp = first->next;
107        YIELD(12);
108        delete first;
109        YIELD(13);
110        if(temp == NULL)
111        {
112            YIELD(14);
113            last = NULL;
114        }
115        else
116        {
117            YIELD(15);
118            temp->prev = NULL;
119        }
120        YIELD(16);
121        first = temp;
122        YIELD(17);
123        lock->Release();
124        return item;
125    }
126 }

```



```
127
128 bool DLLList::IsEmpty()
129 {
130     YIELD(18);
131     if(first == NULL)
132     {
133         YIELD(19);
134         if(last == NULL)
135         {
136             YIELD(20);
137             return true;
138         }
139         DEBUG('e', "[ERROR]first is NULL, last is not NULL!\n");
140         YIELD(21);
141     }
142     else if(last == NULL)
143     {
144         DEBUG('e', "[ERROR]first is not NULL, last is NULL!\n");
145     }
146     YIELD(22);
147     return false;
148 }
149
150 void DLLList::SortedInsert(void *item, int sortKey)
151 {
152     lock->Acquire();
153     YIELD(23);
154     DLLElement *elm = new DLLElement(item, sortKey);
155     YIELD(24);
156     if(IsEmpty())
157     {
158         YIELD(25);
159         first = elm;
160         YIELD(26);
161         last = elm;
162         YIELD(27);
163     }
164     else
165     {
166         YIELD(28);
167         DLLElement *ptr = first;
168         YIELD(29);
```

```

169     while(ptr != NULL && ptr->key < sortKey)
170     {
171         YIELD(30);
172         ptr = ptr->next;
173         YIELD(31);
174     }
175     YIELD(32);
176     if(ptr == first) // insert in the head
177     {
178         YIELD(33);
179         elm->next = first;
180         YIELD(34);
181         first->prev = elm;
182         YIELD(35);
183         first = elm;
184         YIELD(36);
185     }
186     else if(ptr == NULL) // insert in the tail
187     {
188         YIELD(37);
189         elm->prev = last;
190         YIELD(38);
191         last->next = elm;
192         YIELD(39);
193         last = elm;
194         YIELD(40);
195     }
196     else
197     {
198         YIELD(41);
199         elm->next = ptr;
200         YIELD(42);
201         ptr->prev->next = elm;
202         YIELD(43);
203         elm->prev = ptr->prev;
204         YIELD(44);
205         ptr->prev = elm;
206         YIELD(45);
207     }
208 }
209 listEmpty->Signal(lock);
210 lock->Release();

```

```
211 }
212
213 void *DLLList::SortedRemove(int sortKey)
214 {
215     lock->Acquire();
216     while (IsEmpty()) listEmpty->Wait(lock);
217     DLLElement *ptr = first;
218     while(ptr != NULL && ptr->key != sortKey)
219     {
220         ptr = ptr->next;
221     }
222     if(ptr == NULL)
223     {
224         return NULL;
225     }
226     void *item = ptr->item;
227     if(ptr == first)
228     {
229         first = first->next;
230         if(first == NULL) // list is empty
231         {
232             last = NULL;
233         }
234         else
235         {
236             first->prev = NULL;
237         }
238     }
239     else if(ptr == last)
240     {
241         last = last->prev;
242         if(last == NULL) // list is empty
243         {
244             first = NULL;
245         }
246         else
247         {
248             last->next = NULL;
249         }
250     }
251     else
252     {
```

```

253     ptr->next->prev = ptr->prev;
254     ptr->prev->next = ptr->next;
255 }
256 delete ptr;
257 lock->Release();
258 return item;
259 }
260
261 void DLLList::show()
262 {
263     if(IsEmpty())
264     {
265         printf("List is empty!\n");
266     }
267     else
268     {
269         DLLElement *tmp = first;
270         printf("List from head to tail:\n");
271         while(tmp != NULL)
272         {
273             printf("key: %d, item: %p\n", tmp->key, tmp->item);
274             tmp = tmp->next;
275         }
276         tmp = last;
277         printf("List from tail to head:\n");
278         while(tmp != NULL)
279         {
280             printf("key: %d, item: %p\n", tmp->key, tmp->item);
281             tmp = tmp->prev;
282         }
283     }
284 }

```

附件 7 Table.h

```

1  # ifndef TABLE_H
2  # define TABLE_H
3  # include "synch.h"
4  class Table
5  {
6  public:
7      Table(int size);    // Create a table to hold at most size entries.
8      ~Table();
9      int Alloc (void* object); // Allocate a table slot for object,
10                                // returning index of the allocated entry.

```

```

11                                     // Return an error (-1) if no free table
12 slots are available.
13     void* Get (int index); // Retrieve the object from table slot at
14 index,
15                                     // or NULL if not allocated.
16     void Release (int index); // Free the table slot at index.
17 private:
18     int TableSize;
19     void **table;
20     Semaphore* lock;
21 };
22 # endif

```

附件 8 Table.cc

```

1 # include <iostream>
2 # include "Table.h"
3
4 extern Thread *currentThread;
5
6 Table::Table(int size)
7 {
8     TableSize = size;
9     table = new void*[size];
10    for(int i = 0; i < size; ++ i) table[i] = NULL;
11    lock = new Semaphore("TableLock", 1);
12 }
13 Table::~Table()
14 {
15     delete table;
16     delete lock;
17 }
18 int Table::Alloc (void* object)
19 {
20     lock->P();
21     for(int i = 0; i < TableSize; ++ i)
22         if(table[i] == NULL)
23         {
24             table[i] = object;
25             DEBUG('e', "%s: insert %p in %d.\n", currentThread->getName(),
26 object, i);
27             lock->V();
28             return i;
29         }

```

```

30     DEBUG('e', "%s: insert failed because the tabel is full.\n",
31 currentThread->getName());
32     lock->V();
33     return -1;
34 }
35 void* Table::Get (int index)
36 {
37     lock->P();
38     ASSERT(index >= 0 && index < TableSize);
39     if(index >= 0 && index < TableSize && table[index] != NULL)
40     {
41         DEBUG('e', "%s: get %p in %d.\n", currentThread->getName(),
42 table[index], index);
43         lock->V();
44         return table[index];
45     }
46     DEBUG('e', "%s: get NULL in %d because not allocated.\n",
47 currentThread->getName(), index);
48     lock->V();
49     return NULL;
50 }
51 void Table::Release (int index)
52 {
53     lock->P();
54     ASSERT(index >= 0 && index < TableSize);
55     if(index >= 0 && index < TableSize)
56     {
57         DEBUG('e', "%s: release %p in %d.\n", currentThread->getName(),
58 table[index], index);
59         table[index] = NULL;
60     }
61     lock->V();
62 }

```

附件 9 BoundedBuffer.h

```

1 # ifndef BOUNDEDBUFFER_H
2 # define BOUNDEDBUFFER_H
3 #include "synch.h"
4 class BoundedBuffer
5 {
6 public:
7     BoundedBuffer(int maxsize); // Create a bounded buffer to hold at most
8 maxsize bytes.
9     ~BoundedBuffer();

```

```

10     void Read (void* data, int size); // Read size bytes from the buffer,
11     // blocking until enough bytes are available to completely satisfy the
12     request.
13     // Copy the bytes into memory starting at address data
14     void Write (void* data, int size); // Write size bytes into the buffer,
15     // blocking until enough space is available to completely satisfy the
16     request.
17     // Copy the bytes from memory starting at address data .
18 private:
19     char *buffer;
20     int maxSize;
21     int head;
22     int tail;
23     Semaphore* writeLock;
24     Semaphore* readLock;
25     Semaphore* use;
26     Semaphore* remain;
27 };
28 # endif

```

附件 10 BoundedBuffer.cc

```

1 # include "BoundedBuffer.h"
2
3 extern Thread *currentThread;
4 extern void YIELD(int place);
5
6 BoundedBuffer::BoundedBuffer(int maxsize)
7 {
8     buffer = new char[maxsize];
9     maxSize = maxsize;
10    head = tail = 0;
11    readLock = new Semaphore("ReadLock", 1);
12    writeLock = new Semaphore("WriteLock", 1);
13    use = new Semaphore("Use", 0);
14    remain = new Semaphore("Remain", maxsize);
15 }
16
17 BoundedBuffer::~BoundedBuffer()
18 {
19     delete buffer;
20     delete readLock;
21     delete writeLock;
22     delete use;
23     delete remain;

```

```

24 }
25
26 void BoundedBuffer::Read(void *data, int size)
27 {
28     char *out = (char *) data;
29     readLock->P();
30     for(int i = 0; i < size; ++ i)
31     {
32         use->P();
33         out[i] = buffer[head];
34         DEBUG('e', "%s: read %c in %d.\n", currentThread->getName(),
35 out[i], head);
36         head = (head + 1) % maxSize;
37         remain->V();
38     }
39     readLock->V();
40 }
41
42 void BoundedBuffer::Write(void *data, int size)
43 {
44     char *in = (char *) data;
45     writeLock->P();
46     for(int i = 0; i < size; ++ i)
47     {
48         remain->P();
49         buffer[tail] = in[i];
50         DEBUG('e', "%s: write %c in %d.\n", currentThread->getName(),
51 in[i], tail);
52         tail = (tail + 1) % maxSize;
53         use->V();
54     }
55     writeLock->V();
56 }

```

附件 11 threadtest.cc

```

1 #include <stdlib.h>
2 #include <time.h>
3 #include "copyright.h"
4 #include "system.h"
5 #include "dllist.h"
6 #include "Table.h"
7 #include "BoundedBuffer.h"
8
9 extern void insert(int n, DLList &dllist);

```



```
10 extern void remove(int n, DLLlist &dllist);
11 extern void YIELD(int place);
12
13 static int *fargv;
14 static int fargc;
15 static int testNum;
16 static DLLlist dllist;
17 static Table *table;
18 static BoundedBuffer *buffer;
19
20 void ThreadInit(int* argv, int argc)
21 {
22     if(argc > 0)
23     {
24         testNum = argv[0];
25         if(argc > 1)
26         {
27             fargv = argv + 1;
28             fargc = argc - 1;
29         }
30         else
31         {
32             fargv = NULL;
33             fargc = 0;
34         }
35     }
36 }
37
38 int getIntBit(int x)
39 {
40     int n = 0;
41     do ++ n, x /= 10; while(x != 0);
42     return n;
43 }
44
45 void makeRandThread(void(*func)(int arg), char *kind, int id, int
46 MinPriority)
47 {
48     int n1 = strlen(kind), n2 = getIntBit(id);
49     char* name = new char[n1 + n2 + 2];
50
51     name[0] = '\\0';
```

```

52     strcat(name, kind);
53     sprintf(name + n1, " %d", id);
54
55     Thread *thread = new Thread(name, rand() % MinPriority);
56     thread->Fork(func, id);
57
58     DEBUG('e', "%s: make thread %s %d\n", currentThread->getName(),
59 thread->getName(), thread->getPriority());
60 }
61
62 void makeThread(void(*func)(int arg), char *kind, int id, int priority)
63 {
64     int n1 = strlen(kind), n2 = getIntBit(id);
65     char* name = new char[n1 + n2 + 2];
66
67     name[0] = '\0';
68     strcat(name, kind);
69     sprintf(name + n1, " %d", id);
70
71     Thread *thread = new Thread(name, priority);
72     thread->Fork(func, id);
73
74     DEBUG('e', "%s: make thread %s %d\n", currentThread->getName(),
75 thread->getName(), thread->getPriority());
76 }
77
78 void makeThread(void(*func)(int arg), char *kind, int id)
79 {
80     int n1 = strlen(kind), n2 = getIntBit(id);
81     char* name = new char[n1 + n2 + 2];
82
83     name[0] = '\0';
84     strcat(name, kind);
85     sprintf(name + n1, " %d", id);
86
87     Thread *thread = new Thread(name);
88     thread->Fork(func, id);
89
90     DEBUG('e', "%s: make thread %s\n", currentThread->getName(),
91 thread->getName());
92 }
93

```

```
94 void SimpleThread(int which)
95 {
96     int N = fargv[1];
97     DEBUG('e', "%s is running.\n", currentThread->getName());
98     YIELD(60);
99     insert(N, dllist);
100    YIELD(61);
101    remove(N, dllist);
102    YIELD(62);
103    DEBUG('e', "%s is finish.\n", currentThread->getName());
104    dllist.show();
105 }
106
107 void test1()
108 {
109     ASSERT(fargc == 2);
110     int T = fargv[0];
111     srand(time(0));
112     for (int i = 0; i < T; ++ i)
113     {
114         makeThread(SimpleThread, "thread", i);
115     }
116 }
117
118 void dllistInsert(int witch)
119 {
120     DEBUG('e', "%s is running.\n", currentThread->getName());
121     int n1 = fargv[1];
122     insert(n1, dllist);
123     DEBUG('e', "%s is finish.\n", currentThread->getName());
124 }
125
126 void dllistRemove(int witch)
127 {
128     DEBUG('e', "%s is running.\n", currentThread->getName());
129     int n2 = fargv[3];
130     remove(n2, dllist);
131     DEBUG('e', "%s is finish.\n", currentThread->getName());
132 }
133
134 void test2()
135 {
```

```
136     ASSERT(fargc == 4);
137     int t1 = fargv[0], t2 = fargv[2];
138     srand(time(0));
139     for(int i = 0; i < t1; ++ i)
140         makeRandThread(dllistInsert, "Insert", i, 100);
141     for(int i = 0; i < t2; ++ i)
142         makeRandThread(dllistRemove, "Remove", i, 100);
143 }
144
145 void test3()
146 {
147     ASSERT(fargc == 0);
148     table = new Table(3);
149     int index = 0;
150     void *object;
151     // 空表分配
152     index = table->Alloc(new int[1]);
153     ASSERT(index == 0);
154     // 非空表非满表分配
155     index = table->Alloc(new int[1]);
156     ASSERT(index == 1);
157     index = table->Alloc(new int[1]);
158     ASSERT(index == 2);
159     // 满表分配
160     index = table->Alloc(new int[1]);
161     ASSERT(index == -1);
162     // 验证每个表槽与分配时一致
163     for(int i = 0; i < 3; ++ i)
164         table->Get(i);
165     // 每个表槽释放再重新分配
166     for(int i = 0; i < 3; ++ i)
167     {
168         table->Release(i);
169         index = table->Alloc(new int[1]);
170         ASSERT(index == i)
171     }
172     // 全部释放
173     for(int i = 0; i < 3; ++ i)
174         table->Release(i);
175     // 验证每个表槽都释放成功
176     for(int i = 0; i < 3; ++ i)
177     {
```

```
178     object = table->Get(i);
179     ASSERT(object == NULL);
180 }
181 }
182
183 void TableAlloc(int witch)
184 {
185     table->Alloc(new int[1]);
186 }
187
188 void TableGet(int witch)
189 {
190     int size = fargv[0];
191     table->Get(rand() % size);
192 }
193
194 void TableRelease(int witch)
195 {
196     int size = fargv[0];
197     table->Release(rand() % size);
198 }
199
200 void test4()
201 {
202     ASSERT(fargc == 4);
203     int size = fargv[0], n1 = fargv[1], n2 = fargv[2], n3 = fargv[3];
204     table = new Table(size);
205     srand(time(0));
206
207     for (int i = 0; i < n1; ++ i)
208     {
209         makeRandThread(TableAlloc, "Alloc", i, 100);
210     }
211     for (int i = 0; i < n2; ++ i)
212     {
213         makeRandThread(TableGet, "Get", i, 100);
214     }
215     for (int i = 0; i < n3; ++ i)
216     {
217         makeRandThread(TableRelease, "Release", i, 100);
218     }
219 }
```

```

220
221 void Writer(int witch)
222 {
223     int n1 = fargv[2];
224     char *data = new char[n1 + 1];
225
226     for(int i = 0; i < n1; ++ i)
227         data[i] = 'a' + rand() % 26;
228     data[n1] = '\0';
229
230     DEBUG('e', "%s: want to write %s.\n", currentThread->getName(), data);
231     buffer->Write(data, n1);
232 }
233
234 void Reader(int witch)
235 {
236     int n2 = fargv[4];
237     char *data = new char[n2 + 1];
238     data[n2] = '\0';
239
240     buffer->Read(data, n2);
241     DEBUG('e', "%s: read %s.\n", currentThread->getName(), data);
242 }
243
244 void test5()
245 {
246     ASSERT(fargc == 5);
247     int size = fargv[0], t1 = fargv[1], t2 = fargv[3];
248     buffer = new BoundedBuffer(size);
249     srand(time(0));
250     for(int i = 0; i < t1; ++ i)
251     {
252         makeRandThread(Writer, "Writer", i, 100);
253     }
254     for(int i = 0; i < t2; ++ i)
255     {
256         makeRandThread(Reader, "Reader", i, 100);
257     }
258 }
259
260 void thread6(int witch)
261 {

```

```

262     printf("%s %d begin to run.\n", currentThread->getName(),
263 currentThread->getPriority());
264 }
265
266 void test6()
267 {
268     ASSERT(fargc == 1);
269
270     int n = fargv[0];
271     srand(time(0));
272     for(int i = 0; i < n; ++ i)
273         makeRandThread(thread6, "mythread", i, 100);
274 }
275
276 void Writer0(int witch)
277 {
278     int n0 = fargv[2];
279     char *data = new char[3];
280
281     data[2] = '\0';
282     for(int i = 0; i < n0; ++ i)
283     {
284         // data[0] = 'a' + rand() % 26;
285         // data[1] = 'a' + rand() % 26;
286         data[0] = 'a' + i * 2;
287         data[1] = 'a' + i * 2 + 1;
288         DEBUG('e', "%s: want to write %s.\n", currentThread->getName(),
289 data);
290         buffer->Write(data, 2);
291         YIELD(100);
292     }
293 }
294
295 void Writer1(int witch)
296 {
297     int n1 = fargv[4];
298     char *data = new char[3];
299
300     data[2] = '\0';
301     for(int i = 0; i < n1; ++ i)
302     {
303         // data[0] = 'a' + rand() % 26;

```

```

304         // data[1] = 'a' + rand() % 26;
305         data[0] = 'A' + i * 2;
306         data[1] = 'A' + i * 2 + 1;
307         DEBUG('e', "%s: want to write %s.\n", currentThread->getName(),
308 data);
309         buffer->Write(data, 2);
310         YIELD(100);
311     }
312 }
313
314 void Reader0(int witch)
315 {
316     int n2 = fargv[6];
317     char *data = new char[3];
318
319     data[2] = '\0';
320     for(int i = 0; i < n2; ++ i)
321     {
322         buffer->Read(data, 2);
323         DEBUG('e', "%s: read %s.\n", currentThread->getName(), data);
324         YIELD(100);
325     }
326 }
327
328 void test7()
329 {
330     ASSERT(fargc == 7);
331     int size = fargv[0], p0 = fargv[1], p1 = fargv[3], p2 = fargv[5];
332
333     buffer = new BoundedBuffer(size);
334     srand(time(0));
335     makeThread(Writer0, "Writer", 0, p0);
336     makeThread(Writer1, "Writer", 1, p1);
337     makeThread(Reader0, "Reader", 0, p2);
338 }
339
340 void ThreadTest()
341 {
342     switch(testNum)
343     {
344         case 1:
345             test1();

```



```

346         break;
347     case 2:
348         test2();
349         break;
350     case 3:
351         test3();
352         break;
353     case 4:
354         test4();
355         break;
356     case 5:
357         test5();
358         break;
359     case 6:
360         test6();
361         break;
362     case 7:
363         test7();
364         break;
365     default:
366         ASSERT(false);
367         break;
368 }
369 }

```

附件 12 main.cc

```

1  // main.cc
2  // Bootstrap code to initialize the operating system kernel.
3  //
4  // Allows direct calls into internal operating system functions,
5  // to simplify debugging and testing. In practice, the
6  // bootstrap code would just initialize data structures,
7  // and start a user program to print the login prompt.
8  //
9  // Most of this file is not needed until later assignments.
10 //
11 // Usage: nachos -d <debugflags> -rs <random seed #>
12 //        -s -x <nachos file> -c <consoleIn> <consoleOut>
13 //        -f -cp <unix file> <nachos file>
14 //        -p <nachos file> -r <nachos file> -l -D -t
15 //        -n <network reliability> -m <machine id>
16 //        -o <other machine id>
17 //        -z
18 //

```

```
19 // -d causes certain debugging messages to be printed (cf. utility.h)
20 // -rs causes Yield to occur at random (but repeatable) spots
21 // -z prints the copyright message
22 //
23 // USER_PROGRAM
24 // -s causes user programs to be executed in single-step mode
25 // -x runs a user program
26 // -c tests the console
27 //
28 // FILESYS
29 // -f causes the physical disk to be formatted
30 // -cp copies a file from UNIX to Nachos
31 // -p prints a Nachos file to stdout
32 // -r removes a Nachos file from the file system
33 // -l lists the contents of the Nachos directory
34 // -D prints the contents of the entire file system
35 // -t tests the performance of the Nachos file system
36 //
37 // NETWORK
38 // -n sets the network reliability
39 // -m sets this machine's host id (needed for the network)
40 // -o runs a simple test of the Nachos network software
41 //
42 // NOTE -- flags are ignored until the relevant assignment.
43 // Some of the flags are interpreted here; some in system.cc.
44 //
45 // Copyright (c) 1992-1993 The Regents of the University of California.
46 // All rights reserved. See copyright.h for copyright notice and
47 // limitation
48 // of liability and disclaimer of warranty provisions.
49
50 #define MAIN
51 #include "copyright.h"
52 #undef MAIN
53
54 #include "utility.h"
55 #include "system.h"
56 #include "dlist.h"
57
58 #ifdef THREADS
59 extern int testnum;
60 #endif
```

```

61
62 // External functions used by this file
63
64 extern void ThreadTest(void), Copy(char *unixFile, char *nachosFile);
65 extern void Print(char *file), PerformanceTest(void);
66 extern void StartProcess(char *file), ConsoleTest(char *in, char *out);
67 extern void MailTest(int networkID);
68
69 extern void YieldInit(int place);
70 extern void YIELD(int place);
71 extern void ThreadInit(int* argv, int argc);
72
73 //-----
74 // main
75 // Bootstrap the operating system kernel.
76 //
77 // Check command line arguments
78 // Initialize data structures
79 // (optionally) Call test procedure
80 //
81 // "argc" is the number of command line arguments (including the name
82 // of the command) -- ex: "nachos -d +" -> argc = 3
83 // "argv" is an array of strings, one for each command line argument
84 // ex: "nachos -d +" -> argv = {"nachos", "-d", "+"}
85 //-----
86
87 int main(int argc, char **argv)
88 {
89     int argCount; // the number of arguments
90                  // for a particular command
91
92     DEBUG('t', "Entering main");
93     (void)Initialize(argc, argv);
94
95     // #ifdef THREADS
96     for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount)
97     {
98         argCount = 1;
99         if (!strcmp(*argv, "-q"))
100         {
101             while(argc > argCount && argv[argCount][0] != '-')
102                 ++ argCount;

```

```

103         int* fargv = new int[argCount - 1];
104         for(int i = 1; i < argCount; ++ i)
105             fargv[i - 1] = atoi(argv[i]);
106         ThreadInit(fargv, argCount - 1);
107     }
108     else if(!strcmp(*argv, "-y"))
109     {
110         YieldInit(atoi(argv[1]));
111         ++ argCount;
112     }
113 }
114 ThreadTest();
115 // #endif
116
117 for (argc--, argv++; argc > 0; argc -= argCount, argv += argCount)
118 {
119     argCount = 1;
120     if (!strcmp(*argv, "-z")) // print copyright
121         printf(copyright);
122 #ifdef USER_PROGRAM
123     if (!strcmp(*argv, "-x"))
124     { // run a user program
125         ASSERT(argc > 1);
126         StartProcess(*(argv + 1));
127         argCount = 2;
128     }
129     else if (!strcmp(*argv, "-c"))
130     { // test the console
131         if (argc == 1)
132             ConsoleTest(NULL, NULL);
133         else
134         {
135             ASSERT(argc > 2);
136             ConsoleTest(*(argv + 1), *(argv + 2));
137             argCount = 3;
138         }
139         interrupt->Halt(); // once we start the console, then
140                             // Nachos will loop forever waiting
141                             // for console input
142     }
143 #endif // USER_PROGRAM
144 #ifdef FILESYS

```

```

145     if (!strcmp(*argv, "-cp"))
146     { // copy from UNIX to Nachos
147         ASSERT(argc > 2);
148         Copy(*(argv + 1), *(argv + 2));
149         argCount = 3;
150     }
151     else if (!strcmp(*argv, "-p"))
152     { // print a Nachos file
153         ASSERT(argc > 1);
154         Print(*(argv + 1));
155         argCount = 2;
156     }
157     else if (!strcmp(*argv, "-r"))
158     { // remove Nachos file
159         ASSERT(argc > 1);
160         fileSystem->Remove(*(argv + 1));
161         argCount = 2;
162     }
163     else if (!strcmp(*argv, "-l"))
164     { // list Nachos directory
165         fileSystem->List();
166     }
167     else if (!strcmp(*argv, "-D"))
168     { // print entire filesystem
169         fileSystem->Print();
170     }
171     else if (!strcmp(*argv, "-t"))
172     { // performance test
173         PerformanceTest();
174     }
175 #endif // FILESYS
176 #ifdef NETWORK
177     if (!strcmp(*argv, "-o"))
178     {
179         ASSERT(argc > 1);
180         Delay(2); // delay for 2 seconds
181                 // to give the user time to
182                 // start up another nachos
183         MailTest(atoi(*(argv + 1)));
184         argCount = 2;
185     }
186 #endif // NETWORK

```

```
187     }
188
189     currentThread->Finish(); // NOTE: if the procedure "main"
190                             // returns, then the program "nachos"
191                             // will exit (as any other normal program
192                             // would). But there may be other
193                             // threads on the ready list. We switch
194                             // to those threads by saying that the
195                             // "main" thread is finished, preventing
196                             // it from returning.
197     return (0); // Not reached...
198 }
```